

**PENGEMBANGAN APLIKASI PEMBELAJARAN
ALGORITMA DAN STRUKTUR DATA BERBASIS
*LARGE LANGUAGE MODEL***

Laporan Tugas Akhir

Oleh

**Farah Aulia
18222096**



**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

9 Juni 2026

LEMBAR PENGESAHAN

**PENGEMBANGAN APLIKASI PEMBELAJARAN
ALGORITMA DAN STRUKTUR DATA BERBASIS
*LARGE LANGUAGE MODEL***

Laporan Tugas Akhir

Oleh

**Farah Aulia
18222096**

Program Studi Sistem dan Teknologi Informasi
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Laporan Tugas Akhir ini telah disetujui dan disahkan
di Bandung, pada tanggal 9 Juni 2026

Pembimbing

Dr. Riza Satria Perdana, S.T, M.T.

NIP. 19700609 199512 1 002

PERNYATAAN ORISINALITAS

Saya yang bertanda tangan di bawah ini menyatakan dengan sesungguhnya bahwa:

1. Tugas Akhir ini adalah hasil karya saya sendiri yang dibuat dengan sebenarnya sesuai dengan kaidah ilmiah dan etika akademik.
2. Semua sumber rujukan dan data yang saya gunakan dalam penyusunan laporan tugas akhir ini, baik yang berupa kutipan langsung maupun tidak langsung, telah saya cantumkan sumbernya dengan baik dan benar sesuai dengan ketentuan penulisan laporan tugas akhir.
3. Isi laporan ini belum pernah diajukan pada program pendidikan di institusi mana pun.

Apabila di kemudian hari terbukti bahwa laporan ini melanggar hal-hal di atas, saya bersedia menerima sanksi sesuai dengan peraturan yang berlaku di Institut Teknologi Bandung.

Bandung, 9 Juni 2026

Farah Aulia

NIM: 18222096

PERNYATAAN PENGGUNAAN AI

Saya yang bertanda tangan di bawah ini menyatakan bahwa dalam penulisan dokumen Tugas Akhir ini, saya menggunakan alat bantu kecerdasan artifisial (AI) untuk beberapa aspek tertentu di dalam proses penulisan, seperti yang tercantum dalam tabel berikut.

No.	Jenis	Alat Bantu	Tingkat	Tempat
1	Memeriksa ejaan dan tata bahasa	Grammarly	Tinggi	Semua bab
2	Pembuatan teks	Microsoft Copilot	Sedang	Bab II hal. 15
3	Pembuatan grafik, gambar, atau ilustrasi	Tidak ada	Tidak ada	Tidak ada
4	Pencarian informasi atau referensi	...	...	...
5	Penyusunan bahan diskusi	...	...	...
6	Penyusunan kode program	...	...	...
7	Penyusunan formula atau perhitungan matematis	...	...	...
8	Penyusunan konsep desain atau algoritma	...	...	...
9	Penyusunan analisis data	...	...	...
10	Penyusunan kesimpulan atau rekomendasi	...	...	...

Bandung, 9 Juni 2026

Farah Aulia

NIM: 18222096

ABSTRAK

Pengembangan Aplikasi Pembelajaran Algoritma dan Struktur Data Berbasis *Large Language Model*

oleh

Farah Aulia

18222096

Proses manual dalam rantai pasok jeruk di tingkat lapak, yang mencakup sortir, timbang, dan catat, terbukti tidak efisien, memakan waktu, dan rentan terhadap kesalahan. Meskipun solusi berbasis Internet of Things (IoT) dapat mengatasi masalah ini, arsitektur terpusat konvensional berbasis cloud justru menimbulkan masalah baru berupa latensi tinggi yang menghambat alur kerja real-time. Penelitian ini bertujuan untuk merancang dan mengimplementasikan sebuah purwarupa sistem smart scale berbasis IoT dengan pendekatan desentralisasi (edge computing) untuk meminimalkan latensi dan meningkatkan efisiensi operasional. Metode yang digunakan adalah Model Purwarupa, yang diwujudkan dalam bentuk perangkat keras dengan arsitektur prosesor ganda (Raspberry Pi 5 dan ESP32) yang mampu mengintegrasikan fungsi timbang dan analisis kualitas citra secara simultan. Hasil evaluasi menunjukkan bahwa purwarupa dengan arsitektur edge computing yang diimplementasikan mampu memberikan respons 43,5% lebih cepat (latensi rata-rata 18,05 detik) dibandingkan dengan simulasi arsitektur terpusat. Selain itu, sistem ini berhasil meningkatkan efisiensi waktu kerja secara keseluruhan sebesar 58,32% jika dibandingkan dengan metode manual konvensional. Pengujian Penerimaan Pengguna (UAT) yang melibatkan 30 partisipan juga menunjukkan penerimaan yang sangat positif, dengan skor rata-rata untuk kemudahan penggunaan (5,0/5,0), kejelasan antarmuka (4,87/5,0) dan persepsi kinerja (4,95/5,0). Kesimpulannya, implementasi sistem smart scale dengan arsitektur desentralisasi terbukti merupakan solusi yang valid dan efektif untuk menjawab permasalahan latensi dan inefisiensi di lapak. Sistem ini tidak hanya unggul secara teknis dalam hal performa, tetapi juga fungsional dan dapat diterima dengan baik oleh pengguna akhir, serta berpotensi besar untuk modernisasi rantai pasok agribisnis.

Kata kunci: Smart Scale, Internet of Things, Edge Computing, Rantai Pasok Jeruk.

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan Laporan Tugas Akhir yang berjudul “Pengembangan Aplikasi Pembelajaran Algoritma dan Struktur Data Berbasis *Large Language Model*”. Laporan ini disusun sebagai salah satu syarat untuk memperoleh gelar Sarjana pada Program Studi Sistem dan Teknologi Informasi, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung. Pada kesempatan ini, penulis ingin menyampaikan rasa terima kasih yang sebesar-besarnya kepada:

1. Bapak Dr. Riza Satria Perdana, S.T., M.T., selaku dosen pembimbing, atas segala ilmu, bimbingan, arahan, serta masukan yang sangat berharga selama proses pelaksanaan dan penyusunan Tugas Akhir ini.
2. Bapak Dr. Agung Dewandaru, S.T., M.Sc., selaku dosen penguji, atas kritik, saran, dan koreksi yang membangun demi penyempurnaan penelitian ini.
3. Kedua orang tua dan keluarga penulis, atas doa, kasih sayang, dukungan, serta motivasi yang tidak pernah putus kepada penulis.
4. Seluruh dosen dan staf Program Studi Sistem dan Teknologi Informasi ITB, atas ilmu, pengalaman, dan bantuan yang diberikan selama penulis menempuh pendidikan di ITB.
5. Teman-teman penulis, atas kebersamaan, dukungan, semangat, dan bantuan selama proses penyusunan Tugas Akhir ini.

Penulis menyadari bahwa laporan ini masih memiliki keterbatasan dan jauh dari sempurna. Oleh karena itu, penulis menyambut dengan terbuka segala kritik dan saran yang membangun dari berbagai pihak. Semoga laporan ini dapat memberikan manfaat bagi pembaca serta berkontribusi pada pengembangan teknologi pembelajaran berbasis kecerdasan buatan di masa mendatang.

Bandung, Juni 2026

Penulis

Farah Aulia

DAFTAR ISI

LEMBAR PENGESAHAN	iii
PERNYATAAN ORISINALITAS	v
PERNYATAAN PENGGUNAAN AI	vii
ABSTRAK	ix
KATA PENGANTAR	xi
DAFTAR ISI	xiii
DAFTAR GAMBAR	xvii
DAFTAR TABEL	xix
DAFTAR <i>LISTING</i>	xxi
DAFTAR SINGKATAN	xxiii
I PENDAHULUAN	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah	2
I.3 Tujuan	3
I.4 Batasan Masalah	3
I.5 Metodologi	4
I.6 Sistematika Penulisan	5
II STUDI LITERATUR	7
II.1 Pembelajaran Algoritma dan Struktur Data (ASD)	7
II.1.1 Konsep Dasar Algoritma dan Struktur Data	7
II.1.2 Kesulitan Mahasiswa dalam Mempelajari ASD	7
II.2 Large Language Model (LLM)	8
II.2.1 Konsep Dasar dan Mekanisme LLM	8
II.2.2 <i>Prompt Engineering</i>	10
II.3 Evaluasi	11
II.3.1 <i>Black Box Testing</i>	11
II.4 Penelitian Terdahulu	12
II.4.1 <i>Utilizing ChatGPT in a Data Structures and Algorithms Course: A Teaching Assistant's Perspective</i>	12
II.4.2 <i>Evaluating the Effectiveness of Large Language Models in Solving Simple Programming Tasks: A User-Centered Study</i>	12
II.4.3 <i>Leveraging Algorithm Instruction with AI Chatbots: A Detailed Exploration of Their Impact on Students' Computational Thinking</i>	13
III ANALISIS MASALAH	15
III.1 Analisis Kondisi Saat Ini	15
III.2 Analisis Kebutuhan	17
III.2.1 Identifikasi Masalah Pengguna	17
III.2.2 Konsep Dasar Sistem yang Diusulkan	18
III.2.3 Kebutuhan Fungsional	19
III.2.4 Kebutuhan Nonfungsional	21

III.3 Analisis Pemilihan Solusi	22
III.3.1 Alternatif Solusi	22
III.3.1.1 Pemilihan Model LLM	22
III.3.2 Analisis Penentuan Solusi	23
III.3.2.1 Model LLM yang Dipilih	23
III.3.2.2 Pendekatan <i>Guided Learning</i>	24
IV PERANCANGAN	25
IV.1 Gambaran Umum Sistem	25
IV.1.1 Use Case	25
IV.1.2 Sequence Diagram	28
IV.1.3 Arsitektur Sistem	36
IV.2 Perancangan <i>Database</i>	37
IV.2.1 Spesifikasi <i>Platform Database</i>	37
IV.2.2 Desain Skema <i>Database</i>	38
IV.3 Perancangan <i>Website</i>	40
IV.3.1 Perancangan Arsitektur Sistem	40
IV.3.2 Perancangan API	41
IV.3.3 Spesifikasi <i>Hosting</i>	43
V IMPLEMENTASI WEBSITE PEMBELAJARAN ALGORITMA DAN STRUKTUR DATA BERBASIS <i>GUIDED LEARNING</i>	45
V.1 Lingkungan Implementasi	45
V.2 Implementasi <i>Frontend</i>	46
V.2.1 Implementasi Halaman <i>Login</i> dan Registrasi	46
V.2.2 Implementasi Halaman Beranda	48
V.2.3 Implementasi Halaman Topik	50
V.2.4 Implementasi Halaman Materi	51
V.2.5 Implementasi Halaman Contoh	52
V.2.6 Implementasi Halaman Latihan	53
V.2.7 Implementasi Halaman Ringkasan	54
V.2.8 Implementasi Halaman <i>Progress</i>	55
V.2.9 Implementasi Halaman Pengaturan	56
V.3 Implementasi <i>Backend</i>	57
V.3.1 Implementasi Autentikasi	57
V.3.2 Implementasi API Progres	57
V.3.3 Implementasi Mesin Soal Adaptif	58
VI EVALUASI	59
VI.1 Metode Evaluasi	59
VI.1.1 Pengujian <i>Black Box</i>	59
VI.1.2 Pengujian Kegunaan (<i>Usability Testing</i>)	61
VI.1.2.1 <i>Task-Based Usability Test</i>	61
VI.1.2.2 <i>System Usability Scale (SUS)</i>	62
VI.1.2.3 Responden	63
VI.2 Hasil Evaluasi	63
VI.2.1 Hasil Pengujian <i>Black Box</i>	63

VI.2.2 Hasil Pengujian Kegunaan	63
VI.3 Pembahasan Hasil Evaluasi	63
VII PENUTUP	65
VII.1 Kesimpulan	65
VII.2 Saran	65
A KODE PROGRAM	69
A.1 Kode Program <i>Frontend</i>	69
A.2 Kode Program <i>Backend</i>	73
B HASIL SURVEI LAPANGAN	77
B.1 Foto Survei Lokasi 1	77
B.2 Foto Survei Lokasi 2	77
B.3 Transkrip Wawancara dengan Narasumber	77

DAFTAR GAMBAR

II.1	Arsitektur dasar Transformer	9
IV.1	Diagram <i>Use Case</i> Sistem Teman Belajar ASD	26
IV.2	<i>Sequence Diagram</i> FT-01 Login	29
IV.3	<i>Sequence Diagram</i> FT-02 Logout	30
IV.4	<i>Sequence Diagram</i> FT-03 Pilih Topik ASD dan FT-11 <i>Guided Learning Flow</i>	30
IV.5	<i>Sequence Diagram</i> FT-04 Tampilkan Materi, FT-10 <i>Generate LLM</i> , dan FT-14 Adaptasi Konten	31
IV.6	<i>Sequence Diagram</i> FT-05 Tampilkan Contoh, FT-10 <i>Generate LLM</i> , dan FT-14 Adaptasi Konten	32
IV.7	<i>Sequence Diagram</i> FT-06 Latihan Soal, FT-10 <i>Generate LLM</i> , FT-13 <i>Adaptive Engine</i> , dan FT-14 Adaptasi Konten	33
IV.8	<i>Sequence Diagram</i> FT-07 Pembahasan Latihan, FT-12 <i>Feedback Jawaban</i> , dan FT-13 <i>Adaptive Engine</i>	34
IV.9	<i>Sequence Diagram</i> FT-08 Ringkasan Materi	35
IV.10	<i>Sequence Diagram</i> FT-09 Progres Belajar dan FT-13 <i>Adaptive Engine</i>	36
IV.11	Diagram Arsitektur Sistem Teman Belajar ASD	37
IV.12	<i>Entity Relationship Diagram</i> Skema Database	40
V.1	Tampilan Halaman <i>Login</i>	47
V.2	Tampilan Halaman Registrasi	48
V.3	Tampilan Halaman Beranda	49
V.4	Tampilan Halaman Topik	50
V.5	Tampilan Halaman Materi	51
V.6	Tampilan Halaman Contoh	52
V.7	Tampilan Halaman Latihan	53
V.8	Tampilan Halaman Ringkasan	54
V.9	Tampilan Halaman <i>Progress</i>	55
V.10	Tampilan Halaman Pengaturan	56

DAFTAR TABEL

III.1	Kebutuhan Fungsional Sistem Pembelajaran ASD Berbasis LLM . . .	19
III.2	Kebutuhan Nonfungsional Sistem Pembelajaran ASD Berbasis LLM	21
III.3	Perbandingan Model LLM untuk Sistem Pembelajaran ASD	23
IV.1	Deskripsi <i>Use Case</i> Sistem Pembelajaran ASD	26
IV.2	Spesifikasi <i>Platform Database</i>	38
IV.3	Deskripsi Tabel <i>Database</i>	38
IV.4	Komponen Teknologi Sistem Pembelajaran ASD	41
IV.5	Spesifikasi API <i>Endpoint</i> Autentikasi	42
IV.6	Spesifikasi API <i>Endpoint</i> Progres Belajar	42
IV.7	Spesifikasi API <i>Endpoint</i> Latihan	43
IV.8	Spesifikasi <i>Hosting Frontend</i>	43
IV.9	Spesifikasi <i>Hosting Backend</i>	44
IV.10	Spesifikasi <i>Hosting Database</i>	44
V.1	Lingkungan Implementasi Perangkat Keras	45
V.2	Lingkungan Implementasi Perangkat Lunak	46
VI.1	Cakupan Pengujian <i>Black Box</i>	60
VI.2	Interpretasi Skor SUS	62

DAFTAR LISTING

A.1	Implementasi Alur <i>Login</i> dan Penyimpanan Sesi	69
A.2	Fungsi <i>buildInsights</i> untuk Rekomendasi Topik	69
A.3	Perhitungan Progres Topik dari <i>localStorage</i>	70
A.4	Fungsi Penyimpanan Progres Sesi ke <i>localStorage</i> dan <i>Backend</i> .	70
A.5	Mekanisme <i>Toggle</i> Hint dan Pembahasan pada Halaman Contoh . .	71
A.6	Alur <i>Generate</i> Soal Adaptif dan Evaluasi per Soal	71
A.7	Fungsi Perhitungan <i>Streak</i> Belajar Harian	72
A.8	Fungsi <i>refreshSession</i> dan Alur Penyimpanan Perubahan Profil .	72
A.9	Implementasi <i>Password Hashing</i> dan <i>Dependency</i> Autentikasi . . .	73
A.10	Implementasi <i>Upsert</i> Progres Belajar per Topik	73
A.11	Implementasi <i>Endpoint</i> Generate dan Evaluasi Soal Adaptif	74

DAFTAR SINGKATAN

Singkatan	Deskripsi	Pemakaian pertama kali (Bab)
AI	<i>Artificial Intelligence</i> (Kecerdasan Buatan)	I
API	<i>Application Programming Interface</i>	I
ASD	Algoritma dan Struktur Data	I
CI/CD	<i>Continuous Integration / Continuous Deployment</i>	IV
CSS	<i>Cascading Style Sheets</i>	IV
GCP	<i>Google Cloud Platform</i>	IV
GPT	<i>Generative Pre-trained Transformer</i>	I
HTML	<i>HyperText Markup Language</i>	IV
HTTP	<i>HyperText Transfer Protocol</i>	IV
HTTPS	<i>HyperText Transfer Protocol Secure</i>	IV
JSON	<i>JavaScript Object Notation</i>	IV
JWT	<i>JSON Web Token</i>	IV
LLM	<i>Large Language Model</i> (Model Bahasa Besar)	I
ORM	<i>Object-Relational Mapping</i>	IV
PBKDF2	<i>Password-Based Key Derivation Function 2</i>	V
SQL	<i>Structured Query Language</i>	IV

BAB I

PENDAHULUAN

I.1 Latar Belakang

Algoritma dan Struktur Data (ASD) merupakan fondasi utama dalam pendidikan ilmu komputer karena konsep-konsepnya menjadi dasar kemampuan pemecahan masalah, pengembangan perangkat lunak, hingga perancangan sistem berskala besar. Meskipun peran ASD sangat sentral, berbagai studi menunjukkan bahwa mahasiswa konsisten mengalami kesulitan dalam memahami materi ini. Sifat materi ASD yang abstrak, dinamis, dan multidimensional menimbulkan beban kognitif tinggi serta memicu miskonsepsi sejak tahap awal pembelajaran (Mtaho dan Mselle 2024). Kesulitan tersebut kerap bermula dari konsep dasar seperti *pointer* dan struktur data linear, kemudian berkembang menjadi hambatan yang lebih serius ketika mahasiswa mempelajari topik lanjutan seperti *tree*, *graph*, dan analisis kompleksitas algoritma.

Dalam praktiknya, pembelajaran ASD di perguruan tinggi umumnya bertumpu pada modul tertulis, *slide* presentasi, serta penjelasan sinkron dari dosen atau asisten. Pendekatan ini efektif untuk penyampaian konsep secara terstruktur, namun kurang mampu mengakomodasi kebutuhan belajar mandiri mahasiswa di luar jam kuliah. Ketika menghadapi kesulitan secara mandiri, mahasiswa tidak selalu mendapatkan penjelasan tambahan yang segera dan relevan dengan konteks materi yang sedang dipelajari. Keterbatasan umpan balik yang cepat ini berkontribusi pada terbentuknya miskonsepsi yang sulit dikoreksi pada tahap selanjutnya (Leinonen dkk. 2023).

Perkembangan teknologi kecerdasan buatan, khususnya *Large Language Model* (LLM), membuka peluang baru dalam dunia pendidikan. LLM telah menunjukkan kemampuan yang menjanjikan dalam menghasilkan penjelasan konsep, contoh implementasi kode, serta ilustrasi langkah-langkah algoritma secara dinamis dan kontekstual (Kasneji dkk. 2023). Sejumlah penelitian menunjukkan bahwa LLM berpotensi berperan sebagai teman belajar digital (*AI learning companion*) yang mendampingi mahasiswa secara interaktif, memberikan penjelasan sesuai kebutuhan, dan mendukung pembelajaran mandiri yang lebih fleksibel (Kasneji dkk. 2023).

Meskipun demikian, penggunaan LLM secara langsung melalui *chatbot* umum seperti ChatGPT atau Gemini belum cukup efektif sebagai sarana belajar yang terstruktur. Interaksi yang bersifat percakapan bebas membuat penjelasan yang dihasilkan tidak selalu sesuai dengan urutan materi kurikulum. Lebih jauh, LLM berpotensi menghasilkan informasi yang tidak akurat akibat fenomena *hallucination*, serta memberikan jawaban yang tidak konsisten dengan materi yang diajarkan di kelas (Raihan dkk. 2024). Tanpa mekanisme pengarahan yang terstruktur, keluaran LLM sulit dijamin kualitas dan relevansinya untuk keperluan pembelajaran akademik.

Untuk mengatasi keterbatasan tersebut, diperlukan sistem yang mengintegrasikan LLM dalam alur pembelajaran yang terkontrol. Pendekatan *guided learning* menyediakan kerangka sistematis mulai dari pemahaman konsep, eksplorasi contoh, latihan soal, hingga ringkasan materi, sehingga interaksi mahasiswa dengan LLM tetap terarah dan sesuai kurikulum. Sementara itu, teknik *prompt engineering* yang terstruktur memungkinkan keluaran LLM dibatasi pada domain materi yang relevan, sehingga risiko *hallucination* dan inkonsistensi konten dapat diminimalkan (White dkk. 2023). Kombinasi kedua pendekatan ini berpotensi mewujudkan teman belajar digital yang sekaligus interaktif dan terpercaya.

Berdasarkan latar belakang tersebut, penelitian ini mengembangkan Teman Belajar ASD, sebuah aplikasi pembelajaran berbasis web yang memanfaatkan LLM sebagai teman belajar digital untuk mahasiswa. Aplikasi ini menyediakan alur *guided learning* yang terstruktur serta menggunakan *prompt engineering* untuk menjaga relevansi dan konsistensi konten dengan kurikulum ASD. Dengan demikian, diharapkan aplikasi ini dapat membantu mahasiswa memahami konsep ASD secara bertahap, mengurangi miskonsepsi, dan mendukung proses belajar mandiri yang lebih efektif.

1.2 Rumusan Masalah

Secara umum, tugas akhir ini membahas bagaimana merancang dan mengembangkan sebuah aplikasi pembelajaran berbasis web yang memanfaatkan LLM sebagai teman belajar untuk menyajikan materi ASD secara terarah dan sesuai dengan kurikulum. Berdasarkan uraian tersebut, rumusan masalah dalam tugas akhir ini adalah sebagai berikut:

1. Bagaimana merancang dan mengembangkan aplikasi pembelajaran ASD berbasis web dengan alur *guided learning* yang terstruktur dan mudah diikuti oleh mahasiswa, sehingga dapat membantu mengurangi miskonsepsi pada konsep-konsep dasar ASD?

2. Bagaimana memanfaatkan LLM dalam aplikasi sebagai teman belajar digital untuk menghasilkan penjelasan materi, contoh implementasi, dan latihan secara terkontrol agar tetap relevan, akurat, dan konsisten dengan kurikulum?

I.3 Tujuan

Berdasarkan rumusan masalah yang telah ditetapkan, tujuan penelitian ini adalah sebagai berikut:

1. Mengembangkan aplikasi pembelajaran ASD berbasis web dengan alur pembelajaran terstruktur menggunakan pendekatan *guided learning* untuk membantu mahasiswa memahami konsep secara bertahap dan mengurangi miskonsepsi.
2. Mengimplementasikan pemanfaatan LLM dalam aplikasi sebagai teman belajar digital yang mampu menghasilkan penjelasan materi, contoh implementasi, dan latihan secara relevan, akurat, serta konsisten dengan kurikulum.

I.4 Batasan Masalah

Untuk memastikan ruang lingkup penelitian tetap terfokus dan sesuai dengan tujuan yang ingin dicapai, Tugas Akhir ini memiliki batasan-batasan sebagai berikut:

1. Ruang lingkup materi yang digunakan dalam aplikasi dibatasi pada topik-topik dasar ASD seperti *array*, *linked list*, *stack*, *queue*, *tree*, *graph*, *searching*, dan *sorting* sesuai dengan kurikulum yang diberikan dosen.
2. *Large Language Model* (LLM) yang digunakan bersifat model siap pakai (*pre-trained*) dan tidak mencakup proses pelatihan ulang (*fine-tuning*) berskala besar. Penelitian ini hanya memanfaatkan teknik *prompt engineering* dan *retrieval-based guidance* untuk mengarahkan keluaran model.
3. Interaksi pengguna dibatasi melalui mekanisme *guided learning*, sehingga pengguna tidak diberikan akses untuk melakukan percakapan bebas (*open-ended chat*) dengan LLM. Seluruh konten dihasilkan dalam format yang telah ditentukan, seperti penjelasan konsep, contoh kode, latihan soal, dan ringkasan.
4. Sumber referensi yang digunakan LLM dibatasi pada dokumen yang disediakan dosen atau materi kuliah yang relevan. Sistem tidak membahas topik yang berada di luar cakupan kurikulum ASD.
5. Evaluasi penelitian difokuskan pada kualitas konten dan kejelasan penyajian materi, serta aspek kegunaan aplikasi oleh mahasiswa, tanpa mencakup evaluasi performa teknis LLM secara mendalam.

I.5 Metodologi

Metodologi yang digunakan dalam Tugas Akhir ini mengikuti kerangka *Software Development Life Cycle* (SDLC) dengan tahapan berikut:

1. Perencanaan

Tahap ini mencakup identifikasi ruang lingkup penelitian, penentuan tujuan pengembangan aplikasi, serta penyusunan jadwal kerja. Aktivitas meliputi studi literatur terkait kesulitan mahasiswa dalam mempelajari ASD, pendekatan *guided learning*, pemanfaatan LLM dalam pendidikan, serta penetapan tumpukan teknologi dan batasan implementasi yang sesuai dengan waktu pengerjaan.

2. Analisis

Pada tahap ini dilakukan analisis kebutuhan fungsional dan nonfungsional sistem, termasuk identifikasi aktor, alur pembelajaran yang diinginkan, dan kendala yang dialami mahasiswa saat belajar ASD secara mandiri. Analisis juga mencakup pemilihan model LLM yang digunakan (GPT-4o-mini melalui OpenAI API) serta perumusan strategi *prompt engineering* untuk menjaga relevansi dan konsistensi keluaran model terhadap kurikulum.

3. Desain

Tahap desain berfokus pada perancangan arsitektur sistem berlapis (*frontend*, *backend*, dan *database*), model basis data untuk menyimpan data pengguna dan progres belajar, serta alur *guided learning* (materi → contoh → latihan → ringkasan). Selain itu, dirancang pula antarmuka halaman-halaman utama aplikasi dan *template prompt* untuk setiap jenis konten yang dihasilkan LLM.

4. Implementasi

Implementasi dilakukan secara bertahap mengikuti rancangan yang telah disusun. *Frontend* dibangun menggunakan Next.js dengan React, *backend* menggunakan FastAPI berbasis Python, dan basis data menggunakan PostgreSQL. Integrasi LLM dilakukan melalui OpenAI API dengan GPT-4o-mini. Autentikasi pengguna menggunakan JWT, dan seluruh layanan di-*deploy* di Google Cloud Run melalui *pipeline* CI/CD berbasis GitHub Actions.

5. Pengujian

Tahap ini bertujuan untuk memastikan fungsionalitas sistem berjalan sesuai kebutuhan melalui pengujian berbasis skenario yang mencakup uji fungsional dengan pendekatan *black box testing* dan pengujian kebutuhan nonfungsional meliputi aspek *availability*, *security*, dan *compatibility* lintas perangkat dan *browser*. Evaluasi dilakukan pula terhadap kualitas konten yang dihasilkan LLM dan kemudahan pengguna dalam mengikuti alur *guided learning* melalui *task-based usability testing*.

I.6 Sistematika Penulisan

Laporan Tugas Akhir ini disusun secara sistematis agar memudahkan pembaca dalam memahami alur penelitian, perancangan, pengembangan, serta evaluasi aplikasi pembelajaran yang dibangun. Adapun sistematika penulisan dalam laporan ini adalah sebagai berikut.

Bab I Pendahuluan berisi latar belakang, rumusan masalah, tujuan penelitian, batasan masalah, metodologi yang digunakan, serta sistematika penulisan laporan secara keseluruhan.

Bab II Studi Literatur membahas landasan teoritis yang digunakan dalam penelitian ini, meliputi konsep pembelajaran Algoritma dan Struktur Data, karakteristik kesulitan mahasiswa dalam memahami ASD, prinsip kerja *Large Language Model* (LLM), dan evaluasi. Bab ini juga menguraikan penelitian terdahulu yang relevan.

Bab III Analisis membahas kondisi pembelajaran Algoritma dan Struktur Data (ASD) saat ini, termasuk fungsi umum sistem pembelajaran yang digunakan mahasiswa serta berbagai kendala yang masih ditemui. Bab ini juga memuat analisis kebutuhan melalui identifikasi masalah pengguna, perumusan kebutuhan fungsional dan nonfungsional sistem, serta analisis alternatif solusi yang meliputi pemilihan model LLM dan mekanisme *guided learning* yang paling sesuai untuk mendukung pengembangan aplikasi.

Bab IV Perancangan membahas perancangan sistem pembelajaran ASD berbasis web menggunakan LLM, meliputi arsitektur sistem, alur kerja pembelajaran, perancangan basis data, serta visualisasi antarmuka aplikasi yang akan diimplementasikan.

Bab V Implementasi berisi penjelasan tentang proses implementasi solusi yang diusulkan, termasuk langkah-langkah yang diambil, teknologi yang digunakan, dan hasil yang diperoleh.

Bab VI Evaluasi berisi analisis dan evaluasi terhadap solusi yang diimplementasikan, termasuk metode pengujian, hal-hal yang disiapkan sebelum pengujian, hasil pengujian, dan penilaian kinerja.

Bab VII Kesimpulan dan Saran berisi kesimpulan dari hasil pelaksanaan tugas akhir dan saran untuk pengembangan lebih lanjut.

BAB II

STUDI LITERATUR

II.1 Pembelajaran Algoritma dan Struktur Data (ASD)

II.1.1 Konsep Dasar Algoritma dan Struktur Data

Algoritma dan Struktur Data (ASD) merupakan mata kuliah fundamental dalam ilmu komputer karena berfokus pada cara menyusun, menyimpan, dan mengelola data secara efisien. Algoritma dipahami sebagai serangkaian langkah terstruktur untuk menyelesaikan suatu permasalahan, sedangkan struktur data berkaitan dengan cara merepresentasikan dan mengorganisasikan data dalam memori komputer (Cormendk. 2009). Pemahaman terhadap ASD sangat penting karena konsep-konsep ini menjadi dasar bagi pemrograman, optimasi performa program, analisis kompleksitas waktu dan ruang, manajemen memori, hingga perancangan sistem berskala besar (Goodrich dan Tamassia 1998).

Materi ASD umumnya mencakup struktur dasar seperti *array*, *linked list*, *stack*, *queue*, *heap*, *tree*, *graph*, serta algoritma pencarian (*searching*) dan pengurutan (*sorting*). Penguasaan materi ini tidak hanya membutuhkan kemampuan memahami definisi dan contoh, tetapi juga keterampilan menalar alur eksekusi program, memahami perubahan data secara dinamis, dan menganalisis performa algoritma di berbagai kondisi masukan. Tantangan ini sering menjadi penyebab kesulitan belajar mahasiswa dalam memahami konsep ASD, sebagaimana ditunjukkan dalam studi literatur mengenai hambatan umum pembelajaran struktur data dan algoritma (Mtaho dan Mselle 2024).

II.1.2 Kesulitan Mahasiswa dalam Mempelajari ASD

Berbagai penelitian menunjukkan bahwa mahasiswa sering mengalami kesulitan dalam mempelajari Algoritma dan Struktur Data (ASD) karena sifat materinya yang abstrak, dinamis, dan multidimensional. Berdasarkan temuan dari *systematic literature review* yang dilakukan oleh Mtaho dan Mselle (Mtaho dan Mselle 2024), kesulitan terbesar mahasiswa berasal dari beberapa faktor utama, yaitu:

1. Abstraksi konsep yang tinggi, sehingga sulit untuk divisualisasikan. Con-

tohnya adalah konsep *pointer*, *node*, atau relasi antar-*node* pada struktur *tree* dan *graph* yang tidak terlihat secara langsung dalam eksekusi program.

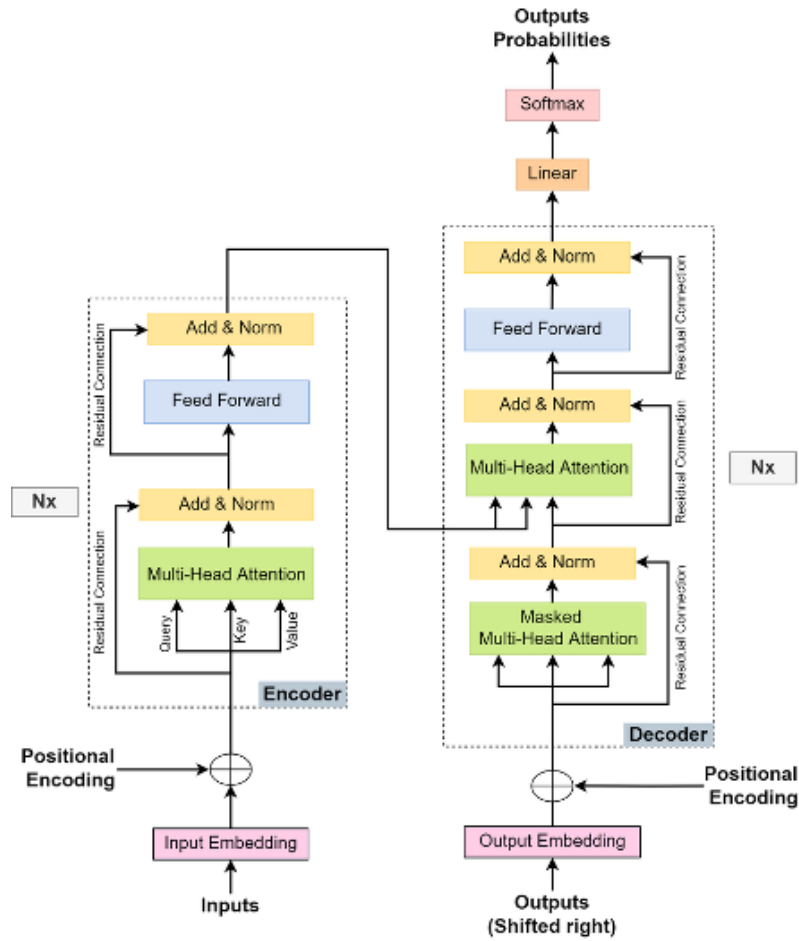
2. Perubahan struktur yang bersifat dinamis, seperti pemutakhiran referensi pada *linked list* atau proses rotasi pada *binary tree*. Perubahan ini menuntut kemampuan visualisasi mental dan penalaran yang kuat untuk memahami bagaimana data bergerak dan dimodifikasi.
3. Beban kognitif yang tinggi, karena pembelajaran ASD mengharuskan mahasiswa memahami representasi struktur data, logika algoritma, serta implementasi kode secara bersamaan. Tuntutan ini sering membuat mahasiswa kewalahan ketika mencoba menghubungkan teori dengan implementasi.
4. Miskonsepsi pada konsep dasar, seperti salah memahami operasi *insert* atau *delete*, cara kerja *pointer*, atau alur eksekusi algoritma. Miskonsepsi tersebut dapat menghambat pemahaman materi lanjutan dan berdampak pada kemampuan pemecahan masalah.

II.2 Large Language Model (LLM)

II.2.1 Konsep Dasar dan Mekanisme LLM

Large Language Model (LLM) merupakan model kecerdasan buatan berskala besar yang dirancang untuk mempelajari struktur bahasa dan memprediksi token berikutnya berdasarkan konteks masukan. Model ini dilatih menggunakan kumpulan data berukuran sangat besar (miliaran token) dan parameter berskala tinggi sehingga mampu melakukan berbagai tugas pemrosesan bahasa secara generatif, seperti menjawab pertanyaan, merangkum teks, menjelaskan konsep, hingga menghasilkan kode program.

Arsitektur utama yang menjadi fondasi LLM modern adalah Transformer (Vaswani dkk. 2017), yang mengandalkan mekanisme *self-attention* untuk memahami hubungan antar-token secara paralel dan efisien. Transformer terdiri dari dua komponen utama, yaitu *encoder* dan *decoder*, tetapi mayoritas LLM generatif saat ini (seperti GPT dan model berbasis *decoder-only* lainnya) hanya menggunakan blok *decoder*. Setiap *decoder layer* terdiri dari beberapa komponen inti yang memproses input secara bertahap.



Gambar II.1 Arsitektur dasar Transformer

Secara umum, proses kerja LLM dapat dijelaskan melalui beberapa tahapan berikut:

1. *Tokenization*: masukan teks diubah menjadi serangkaian token menggunakan metode seperti BPE (*Byte Pair Encoding*) atau *SentencePiece*. Token inilah yang diproses oleh model.
2. *Input Embedding*: setiap token direpresentasikan sebagai vektor berdimensi tinggi (misalnya 512–4096 dimensi), sehingga dapat diproses secara numerik oleh jaringan saraf.
3. *Positional Encoding*: karena Transformer tidak memiliki konsep urutan secara bawaan, informasi posisi ditambahkan menggunakan *positional encoding* sinusoidal atau embedding terlatih lainnya.
4. *Multi-Head Self-Attention*: setiap token menghitung relevansinya terhadap token lain menggunakan tiga vektor utama: Query (Q), Key (K), dan Value (V). Mekanisme perhatian dihitung menggunakan rumus:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Mekanisme ini memungkinkan model mengidentifikasi konteks penting, dependensi jarak jauh, serta memahami relasi antar-token secara paralel melalui beberapa *attention heads*.

5. *Feed-Forward Network (FFN)*: setiap token melewati jaringan *feed-forward* dua lapis dengan aktivasi non-linear. Komponen ini memperluas kapasitas representasi sehingga model mampu melakukan transformasi dan *reasoning* yang lebih kompleks.
6. *Output Softmax Layer*: pada tahap akhir, LLM memprediksi token berikutnya dengan memilih probabilitas tertinggi dari distribusi keluaran. Proses ini dilakukan secara autoregresif hingga seluruh respons terbentuk.

Dengan kombinasi arsitektur Transformer dan pelatihan berskala besar, LLM memiliki kemampuan memahami konteks secara mendalam, menghasilkan teks yang koheren, serta menyesuaikan gaya bahasa sesuai kebutuhan pengguna.

II.2.2 Prompt Engineering

Prompt merupakan masukan teks yang diberikan pengguna untuk memandu model kecerdasan buatan, seperti LLM, dalam menghasilkan keluaran yang relevan dan sesuai tujuan. Dalam *Prompt Engineering*, proses ini melibatkan perancangan *prompt* yang efektif melalui pemilihan instruksi, konteks, contoh, serta batasan tertentu agar model dapat memberikan respons yang lebih akurat, terarah, dan konsisten. Teknik ini menjadi semakin penting karena performa LLM sangat bergantung pada cara instruksi dirumuskan dan seberapa jelas konteks yang diberikan (Minaee 2025; White dkk. 2023).

Berbagai strategi *prompt engineering* telah dikembangkan untuk meningkatkan performa LLM dalam menyelesaikan tugas-tugas kompleks. Beberapa teknik utama yang umum digunakan adalah sebagai berikut:

1. *Zero-Shot Prompting*: model diminta menyelesaikan tugas tanpa contoh sebelumnya, hanya berdasarkan instruksi.
2. *Few-Shot Prompting*: pengguna memberikan beberapa contoh masukan–keluaran untuk membimbing model menghasilkan respons yang serupa.
3. *Chain of Thought (CoT)*: instruksi meminta model menjelaskan proses berpikir secara bertahap, sehingga meningkatkan akurasi pada tugas penalaran.
4. *Tree of Thought (ToT)*: model mengeksplorasi beberapa jalur penalaran, mengevaluasi setiap opsi, kemudian memilih solusi terbaik.
5. *Self-Consistency*: model menghasilkan beberapa jalur penalaran yang kemudian digabungkan untuk memperoleh jawaban paling konsisten.
6. *Reflection Prompting*: setelah memberikan jawaban, model diminta melakukan

evaluasi diri dan memperbaiki respons sebelumnya.

7. *Expert Prompting*: model diarahkan untuk bertindak sebagai seorang pakar dalam domain tertentu agar kualitas penjelasan lebih mendalam dan terstruktur.
8. *Rails / Guardrails*: pemberian batasan dan instruksi khusus agar model tetap fokus pada topik yang relevan serta mengurangi risiko *hallucination*.
9. *Automatic Prompt Engineering (APE)*: menggunakan LLM untuk menghasilkan, menguji, dan memilih *prompt* terbaik secara otomatis.

Teknik-teknik tersebut digunakan untuk meningkatkan kontrol pengguna terhadap keluaran model, mengurangi risiko bias atau kesalahan, serta meningkatkan kualitas penjelasan dan kemampuan penalaran LLM dalam berbagai konteks pembelajaran maupun aplikasi praktis.

II.3 Evaluasi

II.3.1 *Black Box Testing*

Black Box Testing merupakan metode pengujian perangkat lunak yang berfokus pada evaluasi fungsi dan perilaku sistem dari sisi eksternal tanpa melihat struktur internal atau kode program. Pengujian dilakukan dengan memberikan berbagai *input* dan memeriksa apakah *output* yang dihasilkan telah sesuai dengan spesifikasi yang ditentukan (Pressman dan Maxim 2015).

Salah satu teknik *Black Box Testing* yang banyak digunakan adalah *Equivalence Partitioning*, yaitu pendekatan yang membagi domain *input* ke dalam beberapa kelas ekuivalen sehingga hanya satu nilai per kelas yang perlu diuji. Setiap nilai dalam kelas tersebut dianggap mewakili perilaku sistem untuk seluruh anggota kelas. Teknik ini membantu mengurangi jumlah *test case* tanpa menurunkan efektivitas pengujian karena setiap partisi dirancang untuk menangkap potensi kesalahan berdasarkan kelompok *input* yang memiliki karakteristik serupa (Pressman dan Maxim 2015).

Selain itu, pendekatan ini memperkuat efisiensi proses pengujian karena dapat meminimalkan redundansi dan meningkatkan cakupan pengujian. Konsep *Equivalence Partitioning* juga didukung oleh pendekatan lain dalam pengujian perangkat lunak yang menekankan pentingnya representasi kelas nilai uji dalam mendeteksi kesalahan secara sistematis (Myers, Sandler, dan Badgett 2011). Dengan demikian, teknik ini menjadi salah satu metode yang efektif untuk memastikan bahwa fungsi utama perangkat lunak bekerja sesuai dengan spesifikasi yang telah ditetapkan.

II.4 Penelitian Terdahulu

II.4.1 *Utilizing ChatGPT in a Data Structures and Algorithms Course: A Teaching Assistant's Perspective*

Pemanfaatan ChatGPT sebagai alat bantu *Teaching Assistant* (TA) dalam mata kuliah *Data Structures and Algorithms* (DSA) terbukti efektif dalam meningkatkan kualitas pembelajaran. Studi oleh Jamie et al. (et al. 2024) menunjukkan bahwa penggunaan LLM seperti ChatGPT-4o dan ChatGPT-o1 dapat membantu mahasiswa memahami konsep algoritmik kompleks melalui penjelasan terstruktur, pembuatan contoh, dan penyediaan latihan berbasis *structured prompts*. Dalam pendekatan ini, keluaran LLM tetap diverifikasi oleh TA untuk menjaga akurasi dan kesesuaian konten.

Hasil eksperimen terkontrol membandingkan dua kelas—kelas dengan instruksi tradisional dan kelas dengan pendekatan hibrida TA dengan ChatGPT. Kelompok hibrida menunjukkan peningkatan signifikan dengan selisih rata-rata 16,50 poin lebih tinggi pada latihan, kuis, dan ujian. Model AI membantu menjelaskan konsep sulit seperti rekursi, *graph traversal*, serta *dynamic programming*, sembari menyajikan keluaran yang lebih terarah berkat *prompt* yang sistematis.

Namun, penelitian tersebut juga mencatat keterbatasan ChatGPT, seperti ketidakmampuan menghasilkan visualisasi struktur data dan potensi kesalahan pada masalah yang memerlukan kreativitas tinggi. Dengan demikian, peran TA tetap penting untuk memvalidasi keluaran model. Studi ini menegaskan bahwa integrasi LLM tidak menggantikan peran manusia, melainkan memperkuat efektivitas pengajaran melalui kolaborasi manusia dan AI yang terstruktur.

II.4.2 *Evaluating the Effectiveness of Large Language Models in Solving Simple Programming Tasks: A User-Centered Study*

Penelitian oleh Deng (Deng 2025) mengevaluasi efektivitas tiga gaya interaksi ChatGPT-4o, *Passive*, *Proactive*, dan *Collaborative*, dalam membantu pengguna menyelesaikan tugas pemrograman sederhana. Pada mode *Passive*, model hanya merespons permintaan; pada mode *Proactive*, model memberikan saran tambahan; sedangkan mode *Collaborative* memungkinkan diskusi dua arah yang lebih intens.

Sebanyak lima belas siswa sekolah menengah diminta menyelesaikan tiga soal pemrograman di setiap mode interaksi. Hasil eksperimen menunjukkan bahwa mode *Collaborative* menghasilkan waktu penyelesaian tercepat, yaitu rata-rata 2,63 menit per soal, dibandingkan mode *Passive* yang membutuhkan rata-rata 3,60 menit. Hal ini menunjukkan bahwa kualitas interaksi berperan penting dalam efektivitas peng-

gunaan LLM.

Selain peningkatan performa, peserta juga melaporkan tingkat kepuasan dan persepsi kebermanfaatan yang lebih tinggi pada mode *Collaborative*. Temuan ini menegaskan bahwa interaksi yang bersifat dialogis membantu pengguna memahami langkah penyelesaian masalah dengan lebih baik.

II.4.3 *Leveraging Algorithm Instruction with AI Chatbots: A Detailed Exploration of Their Impact on Students' Computational Thinking*

Penggunaan *AI-based chatbots* sebagai pendukung pembelajaran algoritma turut dievaluasi oleh Mahesi et al. (Zaus 2025). Melalui pendekatan *quasi-experimental*, penelitian ini membandingkan kelompok mahasiswa yang belajar dengan bantuan *chatbot* dan kelompok yang menggunakan metode pembelajaran konvensional.

Hasil *pre-test* dan *post-test* menunjukkan peningkatan signifikan pada pemahaman konsep algoritma dan keterampilan *computational thinking* dalam kelompok yang menggunakan *chatbot*. Hal ini terjadi karena *chatbot* menyediakan penjelasan langkah demi langkah, umpan balik otomatis, serta dukungan koreksi kesalahan yang membuat proses belajar lebih interaktif dan mudah diikuti.

Penelitian ini juga menemukan bahwa *chatbot* dapat berfungsi sebagai bentuk *scaffolding*, membantu mahasiswa mengatasi miskonsepsi pada materi yang kompleks, serta menyesuaikan tingkat kesulitan penyampaian materi berdasarkan respons pengguna. Dengan demikian, pengalaman belajar menjadi lebih personal dan adaptif.

BAB III

ANALISIS MASALAH

Bab ini membahas analisis permasalahan yang mendasari pengembangan sistem pembelajaran Algoritma dan Struktur Data (ASD) berbasis *Large Language Model* (LLM) dengan pendekatan *guided learning*. Secara garis besar, bab ini membahas hal-hal berikut:

1. Analisis kondisi pembelajaran ASD saat ini, meliputi fungsi umum sistem yang sudah ada beserta permasalahan yang masih ditemui;
2. Analisis kebutuhan sistem, mencakup identifikasi masalah pengguna, konsep dasar solusi yang diusulkan, serta rumusan kebutuhan fungsional dan non-fungsional; dan
3. Analisis pemilihan solusi, meliputi evaluasi berbagai alternatif teknologi dan penentuan solusi yang paling sesuai untuk mendukung pengembangan sistem.

III.1 Analisis Kondisi Saat Ini

Pembelajaran Algoritma dan Struktur Data (ASD) merupakan salah satu komponen penting dalam kurikulum ilmu komputer yang bertujuan membekali mahasiswa dengan kemampuan memahami struktur data, merancang algoritma, serta menganalisis kompleksitas program. Meskipun demikian, berbagai studi menunjukkan bahwa proses pembelajaran ASD masih menghadapi tantangan signifikan, terutama dalam penyampaian konsep yang bersifat abstrak dan dinamis (Mtaho dan Mselle 2024). Berdasarkan studi literatur dan observasi terhadap praktik pembelajaran saat ini, bagian berikut merangkum fungsi-fungsi utama sistem pembelajaran ASD yang digunakan mahasiswa serta berbagai kendala yang masih ditemui dalam implementasinya.

Adapun fungsi umum yang dimiliki sistem pembelajaran ASD saat ini, yaitu:

1. Menyediakan materi pembelajaran dalam bentuk modul, *slide*, dan contoh kode yang dapat digunakan mahasiswa untuk mempelajari konsep dasar, seperti *array*, *linked list*, *stack*, *queue*, *tree*, *graph*, serta algoritma *searching* dan *sorting*.

2. Menyediakan penjelasan dan ilustrasi visual melalui diagram atau animasi sederhana untuk membantu mahasiswa memahami perubahan struktur data secara bertahap.
3. Memberikan latihan soal dan kuis yang berfungsi menguji pemahaman mahasiswa terhadap konsep algoritmik tertentu.
4. Menyediakan forum diskusi atau sesi tanya jawab dengan dosen dan asisten pengajar untuk membantu menjelaskan konsep yang membingungkan atau memperbaiki miskonsepsi.
5. Menggunakan *environment* pemrograman seperti IDE atau *notebook* untuk memungkinkan mahasiswa mencoba implementasi kode secara langsung.

Meskipun telah menyediakan fitur-fitur tersebut, pendekatan pembelajaran ASD saat ini masih memiliki berbagai permasalahan, antara lain:

1. Sistem pembelajaran umumnya bersifat statis dan tidak adaptif terhadap kebutuhan mahasiswa. Materi, contoh, dan penjelasan yang tersedia biasanya disajikan dalam satu format yang sama sehingga tidak dapat menyesuaikan tingkat pemahaman setiap mahasiswa. Kondisi ini membuat mahasiswa yang mengalami kesulitan pada konsep tertentu tidak mendapatkan penjelasan tambahan yang lebih terarah.
2. Penjelasan mengenai struktur data yang bersifat dinamis, seperti *linked list*, *tree*, atau *graph*, sering kali masih kurang intuitif karena hanya ditampilkan dalam bentuk gambar statis. Hal ini menyulitkan mahasiswa untuk memahami perubahan *pointer*, hubungan antar-*node*, atau proses algoritma secara rinci.
3. Mahasiswa sering mengandalkan penjelasan dari asisten pengajar atau sumber eksternal seperti *chatbot* LLM umum. Namun, penggunaan LLM secara bebas memiliki keterbatasan, seperti ketidakkonsistenan dengan kurikulum atau potensi terjadinya *hallucination* yang dapat menghasilkan informasi keliru.
4. Umpan balik terhadap latihan dan pemahaman konsep biasanya tidak diberikan secara cepat. Mahasiswa harus menunggu sesi asistensi atau waktu koreksi, sehingga proses belajar menjadi kurang responsif dan memperlambat perbaikan miskonsepsi.
5. Sumber referensi materi sering kali tersebar pada berbagai platform seperti LMS, modul digital, forum diskusi, dan sumber eksternal lain, sehingga tidak ada satu wadah terintegrasi yang dapat memandu mahasiswa mengikuti alur belajar secara sistematis.

Saat ini, proses pembelajaran ASD di lingkungan perkuliahan masih sangat bergantung pada metode tradisional yang membutuhkan interaksi langsung dengan dosen

atau asisten pengajar. Meskipun LLM modern seperti ChatGPT, Gemini, atau DeepSeek mulai dimanfaatkan oleh mahasiswa, penggunaannya belum terintegrasi dengan kurikulum dan tidak dibatasi oleh struktur pembelajaran tertentu. Kondisi ini membuat mahasiswa berisiko menerima penjelasan yang tidak sesuai urutan materi atau bahkan bertentangan dengan pendekatan yang diajarkan dalam kelas.

III.2 Analisis Kebutuhan

III.2.1 Identifikasi Masalah Pengguna

Pengguna utama sistem ini adalah mahasiswa yang mempelajari materi Algoritma dan Struktur Data (ASD). Dalam proses pembelajaran saat ini, mahasiswa sering menghadapi berbagai tantangan karena materi ASD memiliki tingkat abstraksi yang tinggi, bersifat dinamis, dan memerlukan kemampuan penalaran algoritmik yang kuat (Mtaho dan Mselle 2024). Metode pembelajaran tradisional yang menggunakan modul, *slide*, serta penjelasan langsung dari dosen dan asisten pengajar belum sepenuhnya mampu menjawab kebutuhan mahasiswa dalam memahami konsep secara mendalam dan bertahap. Kondisi ini menyebabkan mahasiswa sering mengalami miskonsepsi pada konsep dasar, kesulitan mengikuti alur materi, serta kurangnya umpan balik cepat ketika mengalami kebingungan.

Selain itu, mahasiswa kerap mencari bantuan melalui *chatbot* LLM umum, namun interaksi yang tidak terarah dan tidak sesuai materi yang diberikan di kelas berpotensi memberikan penjelasan keliru atau tidak konsisten (White dkk. 2023). Hal ini memperlambat proses belajar dan meningkatkan risiko kesalahan pemahaman. Berikut permasalahan yang dihadapi oleh pengguna tersebut.

1. Mahasiswa saat ini belum memiliki sistem pembelajaran yang menyajikan penjelasan konsep ASD secara terstruktur dan bertahap sehingga memudahkan mereka membangun pemahaman dari dasar hingga tingkat yang lebih kompleks.
2. Mahasiswa tidak memiliki akses terhadap contoh implementasi kode dan ilustrasi langkah demi langkah yang dapat disesuaikan dengan tingkat pemahaman mereka.
3. Mahasiswa belum memiliki sistem pembelajaran yang mampu memberikan penjelasan, contoh, dan latihan yang konsisten dengan materi karena penggunaan LLM umum tidak dibatasi oleh kurikulum atau konteks materi resmi.
4. Mahasiswa tidak mendapatkan peringatan atau umpan balik otomatis ketika terjadi miskonsepsi dalam memahami konsep, seperti kesalahan logika pada struktur data atau salah menafsirkan alur algoritma.
5. Mahasiswa belum memiliki sistem terpadu yang dapat memberikan ringkasan

- materi yang relevan berdasarkan konteks materi yang sedang mereka pelajari.
6. Mahasiswa tidak mendapatkan dukungan untuk mengecek pemahaman mereka secara langsung karena latihan dan evaluasi sering kali tidak memberikan penjelasan balik atau pembahasan detail.
 7. Mahasiswa belum memiliki sistem yang mendukung analisis lebih mendalam melalui visualisasi perubahan struktur data dan simulasi algoritma dalam berbagai skenario untuk memperkuat pemahaman konseptual mereka.

Untuk mencari solusi atas masalah-masalah tersebut, perlu disusun kebutuhan fungsional dan nonfungsional sistem yang diperlukan. Berikut penjabaran mengenai kebutuhan fungsional dan nonfungsional untuk sistem yang akan dibuat.

III.2.2 Konsep Dasar Sistem yang Diusulkan

Konsep dasar sistem pembelajaran yang diusulkan pada tugas akhir ini berfokus pada pemanfaatan *Large Language Model* (LLM) dalam kerangka *guided learning* yang terstruktur untuk membantu mahasiswa memahami materi Algoritma dan Struktur Data (ASD) secara lebih efektif. Sistem ini dirancang untuk mengatasi berbagai permasalahan pembelajaran yang telah diidentifikasi pada bagian sebelumnya, khususnya terkait sifat materi yang abstrak, perlunya penjelasan bertahap, keterbatasan sumber belajar statis, serta ketidakkonsistenan penjelasan ketika menggunakan *chatbot* umum.

Secara konsep, sistem ini berfungsi sebagai pendamping belajar yang memberikan alur pembelajaran terarah meliputi penjelasan konsep, contoh implementasi, latihan soal, dan ringkasan materi. Setiap tahap pembelajaran dihasilkan oleh LLM namun tetap dikendalikan melalui struktur *prompt* dan konteks materi kuliah yang disediakan oleh dosen. Dengan demikian, sistem tidak hanya menghasilkan penjelasan secara generatif, tetapi melakukannya dengan batasan dan struktur yang selaras dengan kurikulum resmi (Minaee 2025).

Selain menyediakan konten terarah, sistem ini juga dirancang untuk memberikan umpan balik otomatis terhadap jawaban mahasiswa pada latihan soal. Dengan adanya umpan balik instan, mahasiswa dapat mengetahui letak kesalahan mereka secara cepat sehingga proses koreksi miskonsepsi dapat berlangsung lebih efektif tanpa harus menunggu sesi asistensi atau evaluasi manual dari pengajar.

Secara keseluruhan, konsep dasar sistem ini adalah menghadirkan platform pembelajaran ASD yang:

1. terstruktur melalui tahapan *guided learning* yang konsisten,
2. adaptif dengan kemampuan LLM menghasilkan penjelasan sesuai konteks

materi dan kebutuhan pengguna,

3. responsif dengan pemberian umpan balik otomatis untuk mendukung koreksi pemahaman, dan
4. terpadu sebagai satu wadah pembelajaran yang menggabungkan materi, contoh, latihan, dan ringkasan secara sistematis.

III.2.3 Kebutuhan Fungsional

Kebutuhan fungsional atau *Functional Requirements* merupakan kebutuhan-kebutuhan dalam bentuk layanan yang diberikan oleh sistem kepada pengguna. Berikut merupakan tabel kebutuhan fungsional dari sistem pembelajaran Algoritma dan Struktur Data berbasis *Large Language Model* dengan pendekatan *guided learning*.

Tabel III.1 Kebutuhan Fungsional Sistem Pembelajaran ASD Berbasis LLM

ID	Kebutuhan	Penjelasan
FR-01	Sistem mampu melakukan proses <i>login</i> sebagai autentikasi pengguna	Sistem menyediakan dialog <i>login</i> yang memvalidasi kredensial pengguna (misalnya akun mahasiswa). Setelah <i>login</i> , sistem memuat profil pengguna, riwayat belajar, serta konfigurasi materi yang relevan dengan pengguna tersebut.
FR-02	Sistem mampu menampilkan alur pembelajaran ASD yang terstruktur	Sistem dirancang untuk menampilkan alur belajar yang terdiri dari tahapan penjelasan konsep, contoh implementasi, latihan soal, dan ringkasan. Pengguna dapat memilih topik materi, lalu sistem menampilkan urutan langkah yang harus diikuti agar pembelajaran menjadi lebih terarah.
FR-03	Sistem mampu menghasilkan penjelasan konsep ASD berbasis LLM sesuai konteks materi	Sistem memanfaatkan LLM dengan struktur <i>prompt</i> yang terarah untuk menghasilkan penjelasan konsep yang relevan, akurat, serta konsisten dengan materi yang sedang dipelajari pengguna.

Bersambung ke halaman berikutnya

Tabel III.1 Kebutuhan Fungsional Sistem Pembelajaran ASD Berbasis LLM (lanjutan)

ID	Kebutuhan	Penjelasan
FR-04	Sistem mampu menampilkan contoh kode dan penjelasan langkah demi langkah	Sistem dapat menghasilkan contoh implementasi kode untuk topik tertentu, disertai penjelasan langkah demi langkah mengenai alur eksekusi algoritma dan perubahan struktur data. Fitur ini membantu pengguna memahami hubungan antara konsep teoretis dan implementasi program.
FR-05	Sistem mampu menyediakan latihan soal dan pembahasan otomatis	Sistem dapat menghasilkan atau menampilkan latihan soal yang terkait dengan materi yang sedang dipelajari pengguna. Setelah pengguna menjawab, sistem memberikan pembahasan otomatis yang menjelaskan alasan jawaban benar maupun salah.
FR-06	Sistem mampu memberikan umpan balik terhadap miskonsepsi pengguna	Sistem menganalisis jawaban atau interaksi pengguna untuk mengidentifikasi pola kesalahan umum, kemudian memberikan umpan balik yang menjelaskan letak miskonsepsi serta mengarahkan pengguna kembali ke penjelasan yang sesuai.
FR-07	Sistem mampu menyajikan ringkasan materi pada akhir sesi belajar	Sistem dapat menghasilkan ringkasan materi yang telah dipelajari pada suatu sesi, termasuk poin penting, istilah kunci, dan hubungan antar konsep. Ringkasan ini membantu pengguna melakukan <i>review</i> dan memperkuat pemahaman konsep.
FR-08	Sistem mampu menyimpan dan menampilkan riwayat serta progres belajar pengguna	Sistem menyimpan data riwayat topik yang sudah dipelajari, latihan yang telah dikerjakan, serta status penyelesaian tiap modul. Pengguna dapat melihat progres belajarnya dan melanjutkan pembelajaran dari posisi terakhir dengan mudah.

Dengan memenuhi kebutuhan fungsional ini, sistem diharapkan dapat memberikan

solusi yang membantu pengguna dalam mengatasi berbagai permasalahan pembelajaran ASD yang telah diidentifikasi sebelumnya.

III.2.4 Kebutuhan Nonfungsional

Kebutuhan nonfungsional atau *Non-Functional Requirements* merupakan kebutuhan yang berisi pernyataan mengenai perilaku dan kualitas yang harus dimiliki oleh sistem pembelajaran Algoritma dan Struktur Data berbasis LLM ke depannya. Berikut merupakan tabel kebutuhan nonfungsional dari sistem pembelajaran ASD berbasis LLM dengan pendekatan *guided learning*.

Tabel III.2 Kebutuhan Nonfungsional Sistem Pembelajaran ASD Berbasis LLM

ID	Parameter	Penjelasan
NF-01	<i>Availability</i>	Sistem harus selalu tersedia dan dapat diakses oleh pengguna tanpa mengalami <i>crash</i> atau gangguan layanan, sehingga proses pembelajaran dapat dilakukan kapan saja tanpa hambatan.
NF-02	<i>Security</i>	Sistem harus melindungi data pengguna seperti progres belajar, riwayat interaksi, serta kredensial akun dengan menerapkan enkripsi dan kontrol akses yang memadai.
NF-03	<i>Compatibility</i>	Sistem harus dapat berjalan pada berbagai perangkat seperti laptop, tablet, maupun <i>smartphone</i> , serta kompatibel dengan beberapa <i>browser</i> modern.
NF-04	<i>Performance</i>	Sistem harus mampu merespons permintaan pengguna dan menghasilkan konten dari LLM dalam waktu yang wajar sehingga tidak mengganggu pengalaman belajar pengguna.
NF-05	<i>Usability</i>	Sistem harus menyediakan antarmuka yang intuitif dan mudah digunakan oleh mahasiswa tanpa memerlukan pelatihan khusus, sehingga pengguna dapat langsung mengikuti alur pembelajaran yang disediakan.

Pemenuhan kebutuhan nonfungsional ini bertujuan untuk memastikan sistem memiliki ketersediaan yang baik, aman digunakan, serta memberikan pengalaman belajar

yang mudah dan intuitif bagi pengguna.

III.3 Analisis Pemilihan Solusi

III.3.1 Alternatif Solusi

Penentuan alternatif solusi dilakukan dengan mempertimbangkan setiap kebutuhan dari permasalahan pembelajaran ASD, khususnya kebutuhan penjelasan yang terarah, konsistensi dengan materi kuliah, serta kemampuan menghasilkan contoh dan latihan secara otomatis. Bagian berikut menguraikan alternatif teknologi yang dapat digunakan dalam pengembangan sistem pembelajaran ASD berbasis LLM, serta pertimbangan pemilihan solusi yang paling sesuai.

III.3.1.1 Pemilihan Model LLM

Terdapat beberapa pilihan model LLM yang populer dalam pengembangan aplikasi edukasi berbasis AI, di antaranya GPT-4o, Gemini 1.5, dan Claude 3. Masing-masing model memiliki kelebihan dan kekurangan dalam hal kemampuan *reasoning*, kualitas penjelasan teknis, serta konsistensi keluaran.

GPT-4o unggul dalam pemahaman konteks yang panjang, penalaran algoritmik, dan kemampuan memberikan penjelasan langkah demi langkah secara stabil. Model ini terbukti efektif untuk menghasilkan contoh kode, menyusun latihan, serta memberikan umpan balik yang terstruktur, menjadikannya kandidat kuat untuk sistem pembelajaran yang memerlukan penjelasan mendalam dalam domain pemrograman. Namun, GPT-4o memerlukan biaya penggunaan yang relatif lebih tinggi dibandingkan beberapa alternatif lainnya.

Di sisi lain, Gemini 1.5 menawarkan kapasitas konteks yang sangat besar sehingga cocok untuk sistem yang memerlukan penyertaan modul kuliah atau dokumen panjang dalam *prompt*. Akan tetapi, kualitas *reasoning* Gemini pada permasalahan algoritmik tertentu kadang kurang stabil dan lebih mudah menghasilkan jawaban yang tidak konsisten.

Claude 3 memiliki kemampuan penjelasan natural dan terstruktur, tetapi terkadang kurang presisi dalam contoh kode yang kompleks, khususnya pada topik algoritma tingkat lanjut.

Perbandingan ketiga model LLM tersebut dirangkum pada Tabel III.3.

Tabel III.3 Perbandingan Model LLM untuk Sistem Pembelajaran ASD

Kriteria	GPT-4o	Gemini 1.5	Claude 3
<i>Reasoning</i> algoritmik	Sangat baik, stabil	Cukup baik, kurang konsisten	Baik, terkadang kurang presisi
Kualitas penjelasan teknis	Mendalam dan sistematis	Cukup baik	Natural, kurang detail pada kode kompleks
Konsistensi keluaran	Tinggi	Sedang	Tinggi
Kapasitas konteks	Besar	Sangat besar	Besar
Biaya penggunaan	Relatif tinggi	Menengah	Menengah
Kesesuaian untuk ASD	Sangat sesuai	Cukup sesuai	Cukup sesuai

III.3.2 Analisis Penentuan Solusi

Berdasarkan hasil evaluasi terhadap berbagai alternatif solusi yang telah diuraikan pada subbab sebelumnya, berikut adalah keputusan pemilihan solusi yang digunakan dalam pengembangan sistem pembelajaran ASD berbasis LLM.

III.3.2.1 Model LLM yang Dipilih

Setelah mempertimbangkan seluruh aspek yang relevan, GPT-4o dipilih sebagai model LLM utama yang digunakan dalam sistem ini. Pemilihan GPT-4o didasarkan pada beberapa pertimbangan berikut:

1. **Kemampuan *reasoning* yang tinggi dan stabil.** GPT-4o menunjukkan konsistensi yang baik dalam menghasilkan penjelasan algoritmik yang akurat, terutama untuk topik-topik yang memerlukan penalaran bertahap seperti proses traversal pada *tree* atau analisis kompleksitas algoritma.
2. **Kualitas contoh kode yang baik.** Model ini mampu menghasilkan contoh implementasi kode dalam berbagai bahasa pemrograman dengan kualitas yang memadai, disertai penjelasan langkah demi langkah yang mudah dipahami.
3. **Dukungan terhadap *prompt engineering* yang terstruktur.** GPT-4o merespons dengan baik terhadap instruksi dan batasan yang diberikan melalui *system prompt*, sehingga memungkinkan sistem untuk mengendalikan format

keluaran sesuai tahapan *guided learning* (White dkk. 2023).

4. **Ekosistem API yang matang.** OpenAI menyediakan API yang stabil dan terdokumentasi dengan baik, mempermudah integrasi dengan *backend* aplikasi.

Dengan pertimbangan tersebut, GPT-4o memberikan keseimbangan terbaik antara kemampuan teknis, stabilitas keluaran, dan kemudahan integrasi untuk mendukung sistem pembelajaran ASD yang terarah, akurat, dan konsisten dengan kurikulum.

III.3.2.2 Pendekatan *Guided Learning*

Selain pemilihan model LLM, pendekatan pembelajaran yang digunakan dalam sistem ini juga merupakan bagian krusial dari solusi yang diusulkan. Pendekatan *guided learning* dipilih karena kemampuannya dalam menyediakan alur belajar yang terstruktur dan terarah, berbeda dengan pendekatan *open-ended chat* yang tidak memiliki batasan konteks. Dengan pendekatan ini, setiap sesi pembelajaran mahasiswa akan mengikuti urutan tahapan yang konsisten, yaitu:

1. **Penjelasan Konsep** : LLM menghasilkan penjelasan teoretis yang disesuaikan dengan topik yang dipilih mahasiswa, mencakup definisi, karakteristik, dan hubungan dengan konsep lain dalam ASD.
2. **Contoh Implementasi** : LLM menyajikan contoh kode beserta penjelasan alur eksekusi secara langkah demi langkah, membantu mahasiswa menghubungkan teori dengan praktik pemrograman.
3. **Latihan Soal** : Sistem menyediakan soal latihan yang relevan dengan materi, disertai pembahasan otomatis setelah mahasiswa menjawab untuk mendeteksi dan mengoreksi miskonsepsi secara langsung.
4. **Ringkasan** : LLM menghasilkan ringkasan poin-poin penting dari sesi belajar yang telah selesai sebagai bahan *review* bagi mahasiswa.

Pendekatan ini memastikan bahwa seluruh konten yang dihasilkan LLM tetap relevan, terstruktur, dan konsisten dengan kurikulum resmi, sekaligus memberikan pengalaman belajar yang lebih terarah dan efektif dibandingkan interaksi bebas dengan *chatbot* umum.

BAB IV

PERANCANGAN

IV.1 Gambaran Umum Sistem

Bagian ini memaparkan gambaran umum sistem Teman Belajar ASD melalui dua model perancangan: *use case diagram* yang mendeskripsikan interaksi antara pengguna dan sistem, serta *sequence diagram* yang menggambarkan urutan komunikasi antar komponen untuk setiap fungsionalitas utama. Kedua model ini menjadi dasar implementasi sistem secara keseluruhan.

IV.1.1 Use Case

Pada sistem aplikasi pembelajaran Algoritma dan Struktur Data berbasis *guided learning* ini, terdapat satu aktor yaitu *user* yang merupakan mahasiswa sebagai pengguna sistem. *User* berinteraksi dengan sistem untuk mempelajari materi ASD secara terstruktur, mulai dari memilih topik, memahami konsep, melihat contoh, mengerjakan latihan, hingga memantau progres belajar.

Sistem dirancang sebagai *teman belajar digital* yang mendampingi mahasiswa dalam proses pembelajaran. Melalui integrasi LLM, sistem mampu memberikan penjelasan materi, contoh implementasi, serta umpan balik terhadap latihan yang dikerjakan oleh *user* secara interaktif dan adaptif.

Berikut merupakan *use case diagram* yang menggambarkan interaksi antara *user* dengan sistem pembelajaran ASD.



Gambar IV.1 Diagram *Use Case* Sistem Teman Belajar ASD

Penjelasan mengenai diagram di atas dan keterkaitannya dengan kebutuhan fungsional dapat dilihat pada Tabel IV.1.

Tabel IV.1 Deskripsi *Use Case* Sistem Pembelajaran ASD

ID	Aktor	Use Case	Deskripsi	Functional Requirements
UC-01	User	Authentication	User melakukan <i>login</i> dengan memasukkan kredensial berupa <i>email</i> dan <i>password</i> untuk mengakses sistem pembelajaran.	FR-01

Bersambung ke halaman berikutnya

Tabel IV.1 Deskripsi *Use Case* Sistem Pembelajaran ASD (lanjutan)

ID	Aktor	Use Case	Deskripsi	Functional Requirements
UC-02	User	Memilih topik ASD	User memilih topik pembelajaran ASD yang ingin dipelajari sebagai langkah awal dalam alur belajar.	FR-02
UC-03	User	Melihat materi	User menerima penjelasan konsep ASD yang dihasilkan sistem secara terstruktur sesuai topik yang dipilih.	FR-03
UC-04	User	Melihat contoh	User melihat contoh implementasi atau ilustrasi dari konsep ASD untuk membantu pemahaman.	FR-04
UC-05	User	Mengerjakan latihan	User mengerjakan soal latihan yang diberikan sistem untuk menguji pemahaman terhadap materi.	FR-05
UC-06	User	Melihat pembahasan	User melihat pembahasan dari latihan yang telah dikerjakan sebagai bentuk umpan balik dari sistem. Relasi <i>extend</i> dari UC-05 menunjukkan bahwa pembahasan hanya tersedia setelah <i>user</i> mengerjakan latihan.	FR-06
UC-07	User	Melihat ringkasan	User melihat rangkuman materi untuk memperkuat pemahaman terhadap konsep yang telah dipelajari.	FR-07
UC-08	User	Melihat <i>progress</i> belajar	User dapat melihat progres pembelajaran yang menunjukkan sejauh mana materi telah dipelajari.	FR-08

Tabel IV.1 menunjukkan bahwa terdapat satu aktor yaitu *user* yang dapat menjalankan berbagai aktivitas untuk memenuhi kebutuhan fungsional sistem. Proses pembelajaran dimulai dari *user* melakukan autentikasi (UC-01), kemudian memilih topik ASD yang ingin dipelajari (UC-02).

Selanjutnya, *user* akan memasuki alur pembelajaran terstruktur dengan melihat materi (UC-03) yang berisi penjelasan konsep yang dihasilkan sistem. Untuk memperdalam pemahaman, *user* dapat melihat contoh implementasi (UC-04) sebelum mengerjakan latihan yang diberikan (UC-05).

Setelah menyelesaikan latihan, *user* dapat melihat pembahasan (UC-06) sebagai bentuk umpan balik dari sistem. Relasi *extend* antara UC-05 dan UC-06 menunjukkan bahwa pembahasan hanya muncul setelah *user* mengerjakan latihan. Selain itu, *user* juga dapat melihat ringkasan materi (UC-07) untuk memperkuat pemahaman, serta memantau progres belajar melalui fitur *progress* (UC-08).

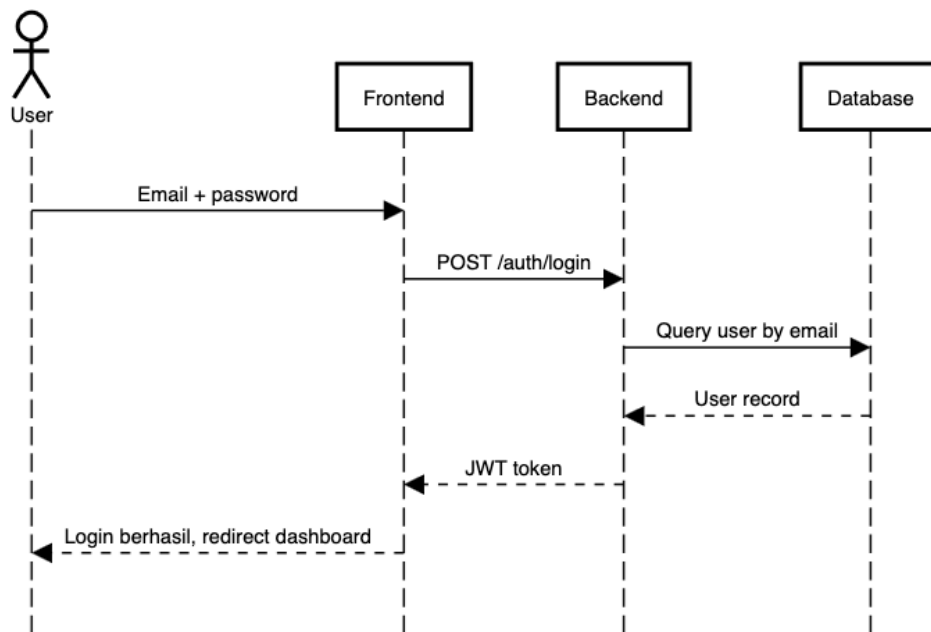
Dengan demikian, seluruh *use case* yang dirancang telah mencerminkan alur *guided learning* yang mendukung pembelajaran ASD secara terstruktur dan interaktif, serta memenuhi kebutuhan fungsional sistem.

IV.1.2 Sequence Diagram

Sequence diagram menggambarkan urutan interaksi antar komponen sistem, *User*, *Frontend*, *Backend*, *Database*, dan OpenAI API, dalam menyelesaikan setiap fungsionalitas. Diagram berikut disusun berdasarkan alur *guided learning* yang telah dirancang, mulai dari autentikasi hingga pemantauan progres belajar.

FT-01 Login

FT-01 Login

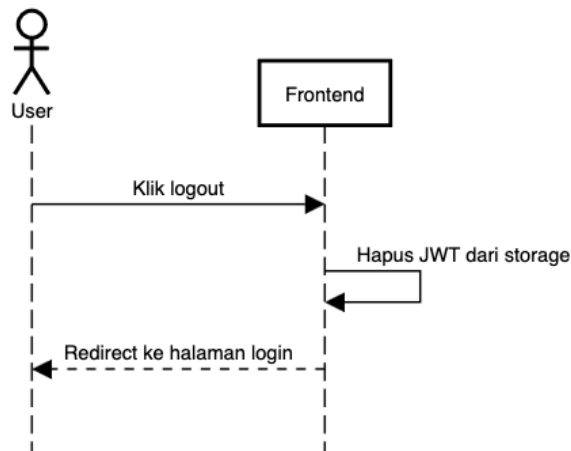


Gambar IV.2 Sequence Diagram FT-01 Login

Diagram pada Gambar IV.2 menggambarkan alur autentikasi pengguna. *User* memasukkan *email* dan *password* pada *frontend*, yang kemudian mengirimkan permintaan POST /auth/login ke *backend*. *Backend* melakukan *query* data pengguna ke *database* berdasarkan *email* yang diberikan. Jika data ditemukan dan kredensial valid, *backend* mengembalikan JWT token ke *frontend*, yang selanjutnya menyimpan *token* tersebut dan mengarahkan *user* ke halaman *dashboard*.

FT-02 Logout

FT-02 Logout

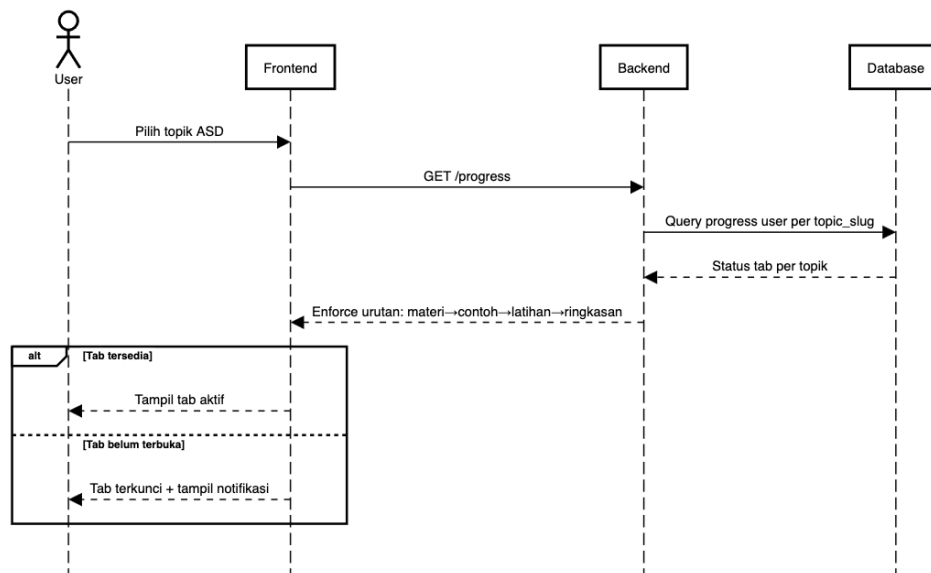


Gambar IV.3 *Sequence Diagram* FT-02 Logout

Diagram pada Gambar IV.3 menggambarkan alur keluar dari sistem. Ketika *user* mengeklik tombol *logout*, *frontend* langsung menghapus *JWT token* dari *storage* lokal tanpa perlu melakukan panggilan ke *backend*. Setelah *token* dihapus, *frontend* mengarahkan *user* kembali ke halaman *login*.

FT-03 Pilih Topik ASD + FT-11 Guided Learning Flow

FT-03 Pilih Topik ASD · FT-11 Guided Learning Flow



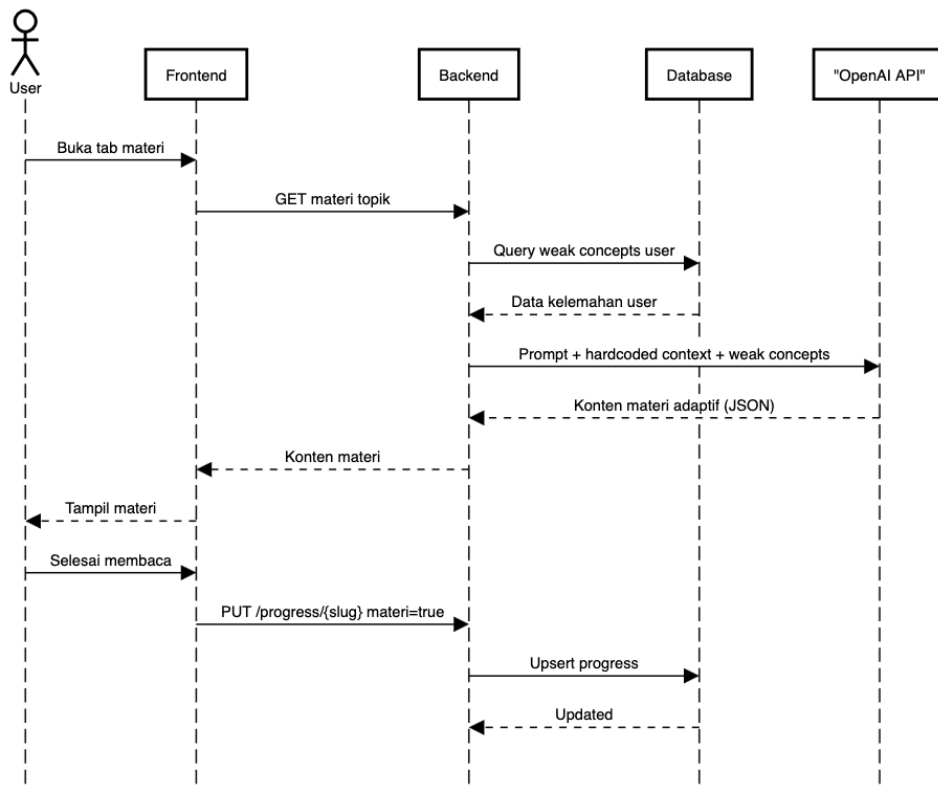
Gambar IV.4 *Sequence Diagram* FT-03 Pilih Topik ASD dan FT-11 *Guided Learning Flow*

Diagram pada Gambar IV.4 menggambarkan alur pemilihan topik ASD sekaligus

penegakan urutan *guided learning*. Setelah *user* memilih topik, *frontend* meminta data progres melalui GET `/progress`. *Backend* mengambil status tab tiap topik dari *database*, lalu menegakkan urutan pembelajaran: materi → contoh → latihan → ringkasan. Jika tab yang ingin diakses sudah tersedia, sistem menampilkan tab tersebut; jika belum terbuka, sistem menguncinya dan menampilkan notifikasi kepada *user*.

FT-04 Tampilkan Materi + FT-10 Generate LLM + FT-14 Adaptasi Konten

FT-04 Tampilkan Materi · FT-10 Generate LLM · FT-14 Adaptasi Konten

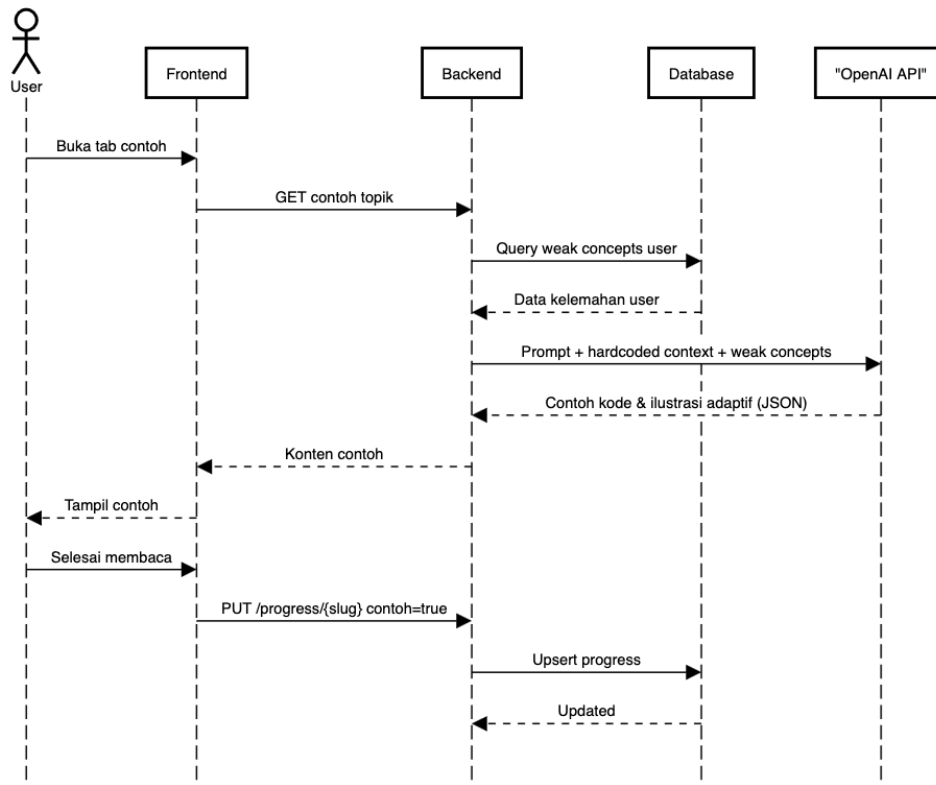


Gambar IV.5 *Sequence Diagram* FT-04 Tampilkan Materi, FT-10 *Generate LLM*, dan FT-14 Adaptasi Konten

Diagram pada Gambar IV.5 menggambarkan alur penampilan materi yang dihasilkan LLM secara adaptif. Ketika *user* membuka tab materi, *frontend* mengirim permintaan ke *backend*. *Backend* terlebih dahulu mengambil data kelemahan konsep *user* (*weak concepts*) dari *database*, kemudian menyusun *prompt* yang menggabungkan konteks materi (*hardcoded context*) dan data kelemahan tersebut sebelum dikirim ke OpenAI API. Respons berupa konten materi adaptif dalam format JSON dikembalikan ke *frontend* dan ditampilkan kepada *user*. Setelah *user* selesai membaca, *frontend* memperbarui progres melalui PUT `/progress/{slug}` dengan status `materi=true`.

FT-05 Tampilkan Contoh + FT-10 *Generate* LLM + FT-14 Adaptasi Konten

FT-05 Tampilkan Contoh · FT-10 *Generate* LLM · FT-14 Adaptasi Konten

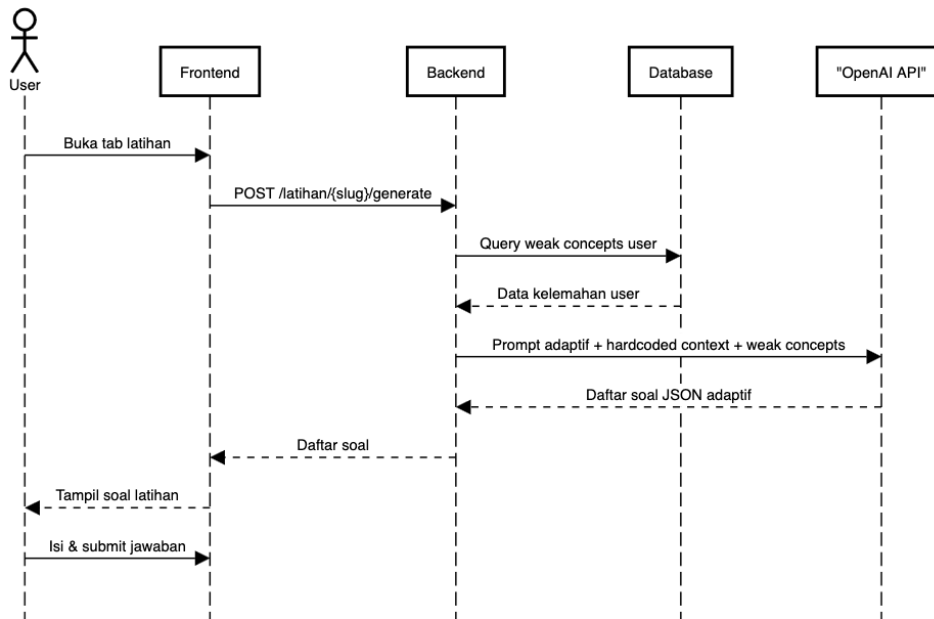


Gambar IV.6 *Sequence Diagram* FT-05 Tampilkan Contoh, FT-10 *Generate* LLM, dan FT-14 Adaptasi Konten

Diagram pada Gambar IV.6 menggambarkan alur penampilan contoh implementasi yang bersifat adaptif. Alur ini serupa dengan FT-04, namun keluaran yang dihasilkan OpenAI API berupa contoh kode dan ilustrasi langkah demi langkah yang disesuaikan dengan kelemahan konsep *user*. Setelah *user* selesai mempelajari contoh, progres diperbarui dengan status `contoh=true` sehingga tab latihan dapat dibuka.

FT-06 Latihan Soal + FT-10 Generate LLM + FT-13 Adaptive Engine + FT-14 Adaptasi Konten

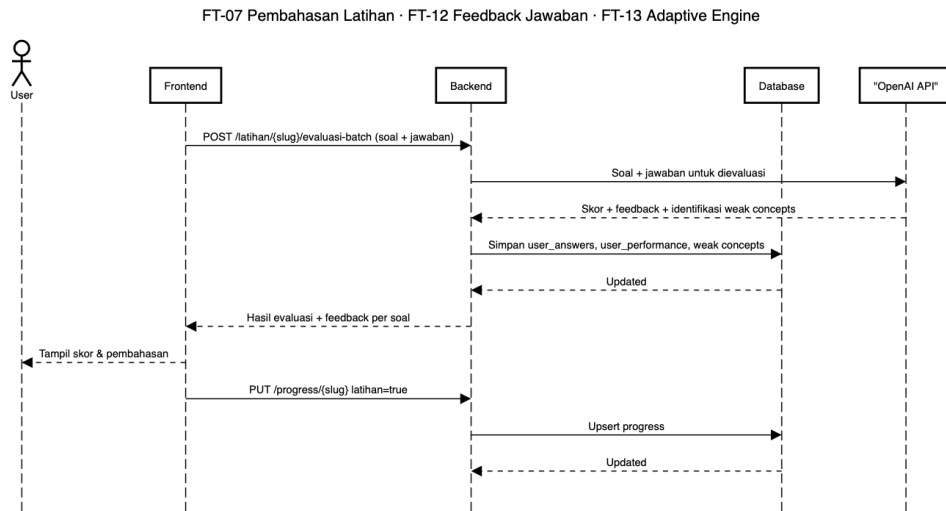
FT-06 Latihan Soal · FT-10 Generate LLM · FT-13 Adaptive Engine · FT-14 Adaptasi Konten



Gambar IV.7 *Sequence Diagram* FT-06 Latihan Soal, FT-10 *Generate LLM*, FT-13 *Adaptive Engine*, dan FT-14 *Adaptasi Konten*

Diagram pada Gambar IV.7 menggambarkan alur pembuatan dan penyajian latihan soal secara adaptif. *Frontend* mengirim permintaan `POST /latihan/{slug}/generate` ke *backend*. *Backend* mengambil data *weak concepts* dari *database* dan menyusun *prompt* adaptif yang memprioritaskan konsep-konsep yang masih lemah bagi *user*. *OpenAI API* menghasilkan daftar soal dalam format JSON yang kemudian ditampilkan kepada *user*. *User* mengisi jawaban dan melakukan *submit* untuk dievaluasi pada tahap berikutnya.

FT-07 Pembahasan Latihan + FT-12 *Feedback* Jawaban + FT-13 *Adaptive Engine*

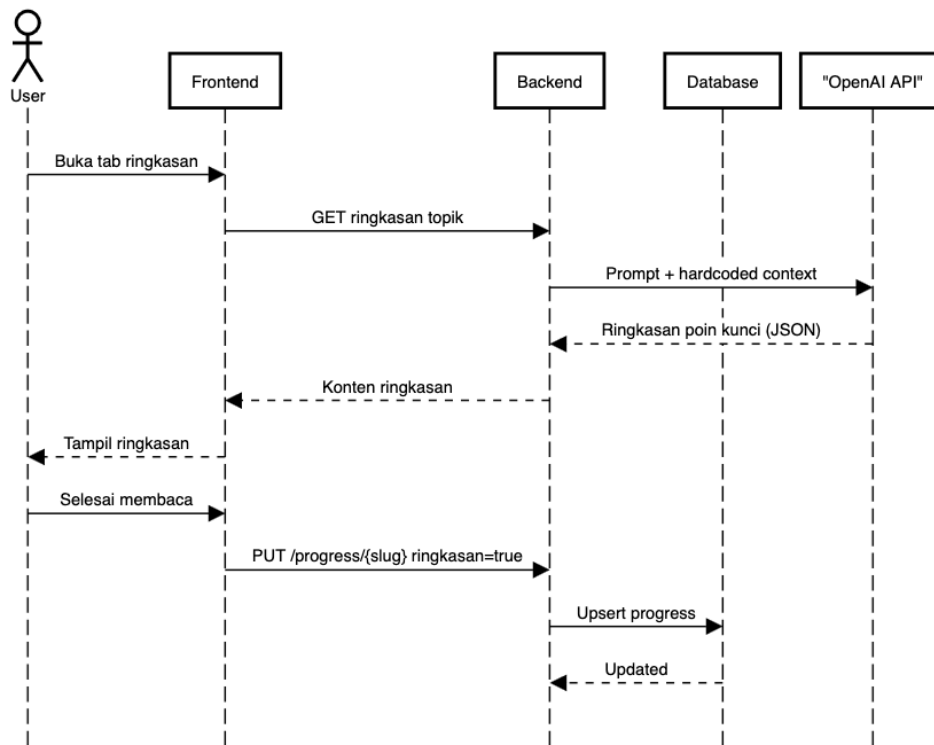


Gambar IV.8 *Sequence Diagram* FT-07 Pembahasan Latihan, FT-12 *Feedback* Jawaban, dan FT-13 *Adaptive Engine*

Diagram pada Gambar IV.8 menggambarkan alur evaluasi jawaban dan pembaruan model adaptif. Setelah *user* mengirim jawaban, *frontend* meneruskan seluruh pasangan soal dan jawaban ke *backend* melalui POST `/latihan/{slug}/evaluasi-batch`. *Backend* mengirimkan data tersebut ke OpenAI API untuk dievaluasi, dan API mengembalikan skor, *feedback* per soal, serta identifikasi *weak concepts* baru. *Backend* menyimpan hasil evaluasi, termasuk *user_answers*, *user_performance*, dan *weak concepts*, ke *database* untuk digunakan sebagai dasar adaptasi pada sesi berikutnya. *Frontend* kemudian menampilkan skor dan pembahasan kepada *user*, dan memperbarui progres dengan status `latihan=true`.

FT-08 Ringkasan Materi

FT-08 Ringkasan Materi

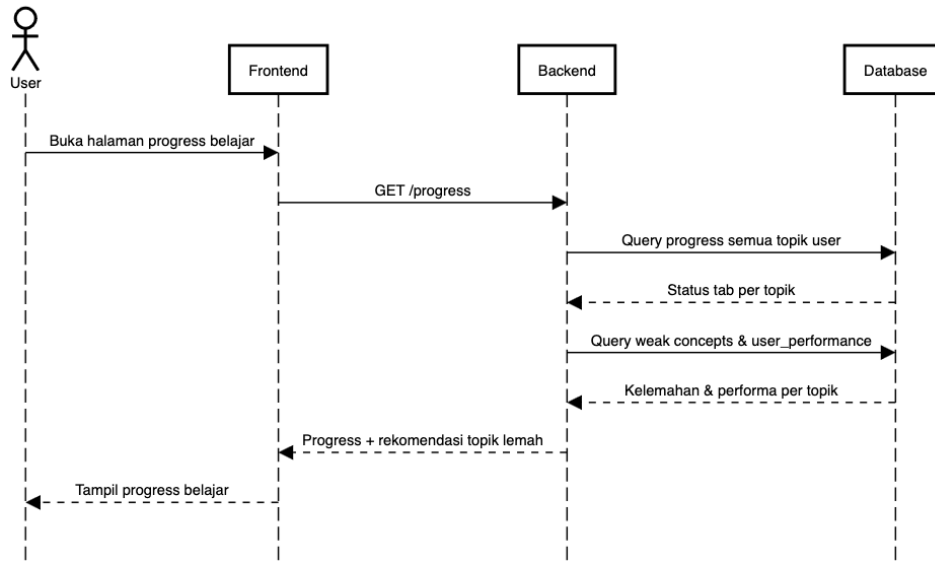


Gambar IV.9 *Sequence Diagram* FT-08 Ringkasan Materi

Diagram pada Gambar IV.9 menggambarkan alur penampilan ringkasan materi di akhir sesi belajar. *Frontend* mengirim permintaan GET ringkasan ke *backend*, yang kemudian meneruskan *prompt* beserta konteks materi ke OpenAI API. API menghasilkan ringkasan poin-poin kunci dalam format JSON. Setelah *user* selesai membaca ringkasan, progres diperbarui dengan status `ringkasan=true`, menandakan bahwa *user* telah menyelesaikan seluruh tahapan pembelajaran untuk topik tersebut.

FT-09 Progres Belajar + FT-13 Adaptive Engine

FT-09 Progress Belajar · FT-13 Adaptive Engine



Gambar IV.10 *Sequence Diagram* FT-09 Progres Belajar dan FT-13 *Adaptive Engine*

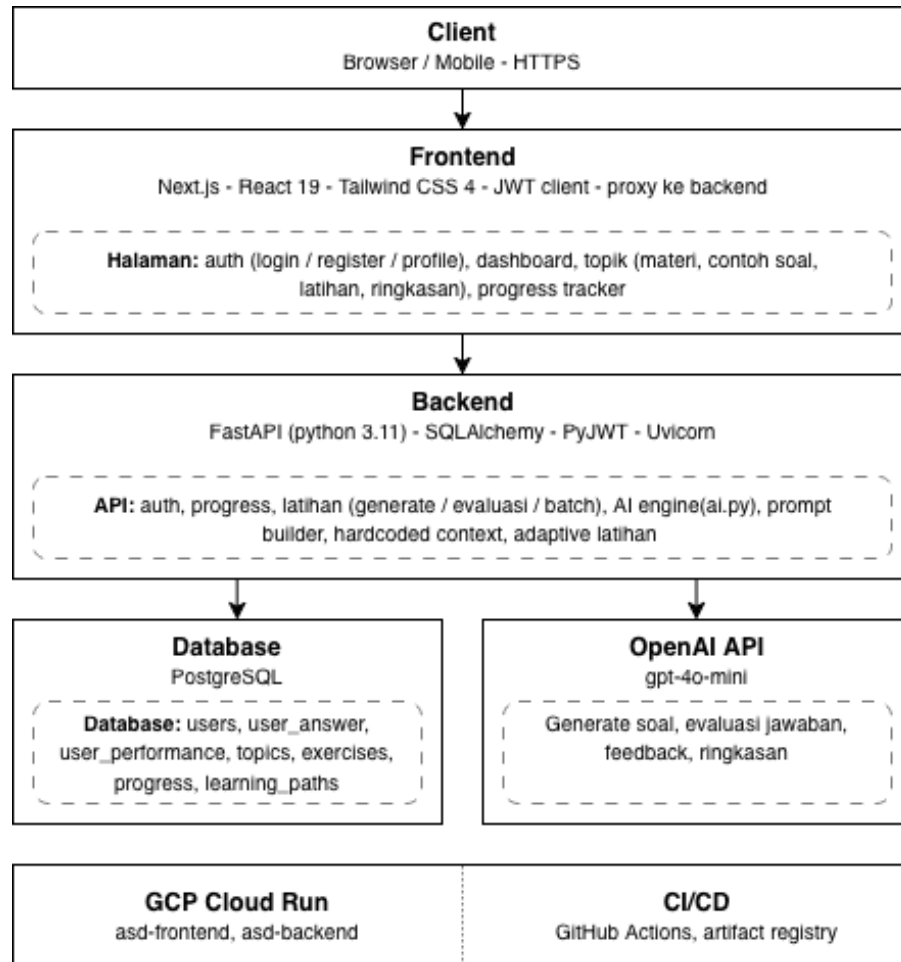
Diagram pada Gambar IV.10 menggambarkan alur penampilan halaman progres belajar yang terintegrasi dengan *adaptive engine*. *Frontend* meminta data melalui GET /progress, dan *backend* melakukan dua *query* ke *database*: pertama untuk mengambil status tab seluruh topik, dan kedua untuk mengambil data *weak concepts* serta performa *user* per topik. *Backend* menggabungkan kedua data tersebut dan mengembalikan informasi progres lengkap beserta rekomendasi topik yang perlu diperkuat. *Frontend* menampilkan seluruh informasi tersebut kepada *user* dalam satu halaman yang komprehensif.

IV.1.3 Arsitektur Sistem

Sistem Teman Belajar ASD dirancang menggunakan arsitektur berlapis yang terdiri atas empat komponen utama: *Client*, *Frontend*, *Backend*, serta *Database* dan OpenAI API. *Client* mengakses sistem melalui *browser* atau perangkat *mobile* via HTTPS. *Frontend* yang dibangun dengan Next.js berperan sebagai antarmuka pengguna sekaligus *proxy* ke *backend*, menangani halaman autentikasi, *dashboard* topik, alur pembelajaran, dan *progress tracker*. *Backend* berbasis FastAPI mengelola seluruh logika bisnis meliputi autentikasi, progres belajar, generasi soal latihan, evaluasi jawaban, dan *adaptive engine*. Data pengguna disimpan di PostgreSQL, sedangkan generasi konten pembelajaran dilakukan melalui OpenAI API (GPT-4o-mini). Seluruh layanan di-deploy di GCP Cloud Run dengan *pipeline* CI/CD melalui GitHub

Actions.

Gambar IV.11 menunjukkan diagram arsitektur sistem secara keseluruhan.



Gambar IV.11 Diagram Arsitektur Sistem Teman Belajar ASD

IV.2 Perancangan Database

Bagian ini menjelaskan perancangan *database* yang digunakan oleh sistem, mencakup pemilihan platform, spesifikasi teknis, serta desain skema tabel beserta relasi antar-entitasnya. Perancangan *database* menjadi fondasi penyimpanan data pengguna, progres belajar, dan mekanisme adaptif sistem.

IV.2.1 Spesifikasi Platform Database

PostgreSQL dipilih sebagai sistem manajemen *database* karena bersifat *open source*, mendukung tipe data relasional yang kaya, serta memiliki ekosistem yang matang untuk pengembangan aplikasi web. Akses ke *database* dilakukan melalui SQLAlchemy sebagai ORM (*Object-Relational Mapper*) dengan *driver* psycopg3, sehingga

interaksi antara *backend* Python dan *database* dapat dilakukan secara efisien dan aman. Tabel IV.2 menyajikan spesifikasi *platform database* yang digunakan.

Tabel IV.2 Spesifikasi *Platform Database*

Aspek	Spesifikasi
<i>Database Engine</i>	PostgreSQL
<i>Driver</i>	psycopg (psycopg3)
ORM	SQLAlchemy
Nama <i>Database</i>	asd_learning_db
Port	5432 (default)

IV.2.2 Desain Skema *Database*

Skema *database* terdiri atas sepuluh tabel yang dirancang untuk mendukung fungsionalitas autentikasi, konten pembelajaran, pencatatan aktivitas pengguna, dan mekanisme adaptif. Tabel IV.3 menyajikan deskripsi singkat setiap tabel beserta kolom-kolom utamanya.

Tabel IV.3 Deskripsi Tabel *Database*

Tabel	Kolom Utama	Fungsi
users	user_id, name, email, password, google_id, created_at	Menyimpan data akun pengguna, mendukung autentikasi <i>password</i> dan OAuth Google
topics	topic_id, topic_name, description, difficulty_level	Menyimpan daftar topik ASD beserta deskripsi dan tingkat kesulitan
materials	material_id, topic_id, content, generated_by_llm	Menyimpan konten materi per topik, dapat dihasilkan oleh LLM
examples	example_id, topic_id, content, generated_by_llm	Menyimpan contoh implementasi per topik, dapat dihasilkan oleh LLM

Bersambung ke halaman berikutnya

Tabel IV.3 Deskripsi Tabel *Database* (lanjutan)

Tabel	Kolom Utama	Fungsi
exercises	exercise_id, topic_id, question, difficulty_level, generated_by_llm	Menyimpan soal latihan per topik, dapat dihasilkan oleh LLM
explanations	explanation_id, exercise_id, content, generated_by_llm	Menyimpan pembahasan soal latihan; berelasi <i>one-to-one</i> dengan exercises
user_answers	answer_id, user_id, exercise_id, answer_text, is_correct, score, created_at	Mencatat jawaban pengguna terhadap soal latihan beserta skor dan status kebenaran
progress	progress_id, user_id, topic_slug, materi, contoh, latihan, ringkasan, last_accessed	Menyimpan status penyelesaian tiap tahapan <i>guided learning</i> per pengguna per topik
user_performance	performance_id, user_id, topic_id, accuracy, avg_score, weakness_level	Menyimpan metrik performa pengguna per topik sebagai dasar <i>adaptive engine</i>
learning_paths	path_id, topic_id, step_order, step_type	Mendefinisikan urutan tahapan pembelajaran per topik

Gambar IV.12 menunjukkan *Entity Relationship Diagram* (ERD) yang menggambarkan seluruh tabel dan relasi antar-tabel pada sistem.



Gambar IV.12 *Entity Relationship Diagram* Skema Database

IV.3 Perancangan Website

Bagian ini membahas perancangan teknis aplikasi web Teman Belajar ASD, meliputi arsitektur sistem dan tumpukan teknologi yang digunakan, spesifikasi *endpoint* API, rancangan antarmuka pengguna, serta konfigurasi *hosting* dan *deployment*.

IV.3.1 Perancangan Arsitektur Sistem

Aplikasi Teman Belajar ASD dibangun menggunakan arsitektur *client-server* yang terdiri dari tiga lapisan utama: *frontend*, *backend*, dan *database*. Komponen *frontend* dibangun dengan Next.js dan berkomunikasi dengan *backend* melalui RESTful API berbasis FastAPI. *Backend* menangani logika bisnis, autentikasi JWT, serta integrasi dengan OpenAI API untuk menghasilkan konten pembelajaran secara adaptif.

Data pengguna dan progres belajar disimpan pada PostgreSQL. Tabel IV.4 menyajikan komponen teknologi yang digunakan pada setiap lapisan sistem.

Tabel IV.4 Komponen Teknologi Sistem Pembelajaran ASD

Komponen	Teknologi	Fungsi
<i>Framework Frontend</i>	Next.js 16	<i>Server-side rendering</i> dan <i>routing</i> halaman
<i>UI Library</i>	React 19	Komponen antarmuka pengguna
<i>Styling</i>	Tailwind CSS 4	<i>CSS framework</i> berbasis utilitas
<i>Framework Backend</i>	FastAPI	RESTful API dan logika bisnis
<i>Application Server</i>	Uvicorn	ASGI <i>server</i> untuk menjalankan FastAPI
ORM	SQLAlchemy	<i>Object-Relational Mapping</i> untuk akses <i>database</i>
<i>Database Driver</i>	psycopg3	Koneksi Python ke PostgreSQL
<i>Database</i>	PostgreSQL	Penyimpanan data pengguna dan progres belajar
Autentikasi	PyJWT	Pembuatan dan validasi JWT <i>token</i>
<i>AI Integration</i>	OpenAI API (GPT-4o-mini)	Generasi konten materi, contoh, latihan, dan evaluasi adaptif
<i>HTTP Client</i>	httpx	Komunikasi asinkron ke OpenAI API

IV.3.2 Perancangan API

Frontend berkomunikasi dengan *backend* melalui RESTful API yang dibangun menggunakan *framework* FastAPI. Seluruh *endpoint* menggunakan *prefix* /api dan bertukar data dalam format JSON. Akses ke *endpoint* yang memerlukan autentikasi dilakukan dengan menyertakan JWT *token* pada *header* Authorization: Bearer <token>. API dikelompokkan menjadi tiga kelompok fungsional, yaitu autentikasi, progres belajar, dan latihan.

Tabel IV.5 menyajikan spesifikasi *endpoint* autentikasi yang menangani proses registrasi, *login*, pengambilan data pengguna aktif, dan pembaruan profil.

Tabel IV.5 Spesifikasi API *Endpoint* Autentikasi

Metode	Endpoint	Body / Header	Fungsi
POST	/api/auth/register	name, email, password	Mendaftarkan pengguna baru dan mengembalikan JWT <i>token</i>
POST	/api/auth/login	email, password	Melakukan autentikasi dan mengembalikan JWT <i>token</i>
GET	/api/auth/me	Header: Authorization	Mengambil data pengguna yang sedang aktif
PUT	/api/auth/profile	name, email, current_password, new_password	Memperbarui nama, <i>email</i> , atau kata sandi pengguna

Tabel IV.6 menyajikan spesifikasi *endpoint* progres belajar yang digunakan untuk membaca dan memperbarui status penyelesaian tiap tahapan pembelajaran per topik.

Tabel IV.6 Spesifikasi API *Endpoint* Progres Belajar

Metode	Endpoint	Body / Header	Fungsi
GET	/api/progress	Header: Authorization	Mengambil seluruh data progres belajar milik pengguna
PUT	/api/progress/{topic_slug}	materi, contoh, latihan, ringkasan	Memperbarui status penyelesaian tab pembelajaran untuk satu topik

Tabel IV.7 menyajikan spesifikasi *endpoint* latihan yang digunakan untuk menghasilkan soal berbasis LLM secara adaptif serta mengevaluasi jawaban pengguna baik secara individual maupun sekaligus dalam satu *batch*.

Tabel IV.7 Spesifikasi API *Endpoint* Latihan

Metode	Endpoint	Body	Fungsi
POST	/api/latihan/ {topic_slug}/generat	jumlah, kelemahan, tipe_paksa, topik_referensi	Menghasilkan soal latihan adaptif berbasis LLM sesuai topik dan kelemahan pengguna
POST	/api/latihan/ {topic_slug}/evaluasi	soal, jawaban	Mengevaluasi satu jawaban latihan dan mengembalikan <i>feed-back</i>
POST	/api/latihan/ {topic_slug}/ evaluasi-batch	soal[] , jawaban{}	Mengevaluasi seluruh jawaban sekaligus dan memperbarui data <i>weak concepts</i> pengguna

IV.3.3 Spesifikasi *Hosting*

Aplikasi di-*deploy* menggunakan Google Cloud Run sebagai *platform hosting* berbasis kontainer yang bersifat *serverless*. *Frontend* dan *backend* masing-masing dimasukkan dalam *Docker image* dan dijalankan sebagai layanan Cloud Run yang terpisah. *Database* menggunakan Google Cloud SQL for PostgreSQL yang terhubung ke *backend* melalui Cloud SQL Connector. Seluruh proses *build* dan *deployment* dijalankan secara otomatis melalui GitHub Actions setiap kali terdapat *push* ke *branch* utama.

Tabel IV.8 menyajikan spesifikasi *hosting* untuk layanan *frontend*.

Tabel IV.8 Spesifikasi *Hosting Frontend*

Aspek	Spesifikasi
<i>Platform</i>	Google Cloud Run (<i>serverless container</i>)
<i>Runtime</i>	Node.js 20 Alpine
<i>Framework</i>	Next.js (output: standalone)
<i>Build Command</i>	npm run build
<i>Container Port</i>	3000
<i>Region</i>	asia-southeast2 (Jakarta)
<i>Deployment Method</i>	GitHub Actions (<i>Continuous Deployment</i>)
<i>Container Registry</i>	Google Artifact Registry

Tabel IV.9 menyajikan spesifikasi *hosting* untuk layanan *backend*.

Tabel IV.9 Spesifikasi *Hosting Backend*

Aspek	Spesifikasi
<i>Platform</i>	Google Cloud Run (<i>serverless container</i>)
<i>Runtime</i>	Python 3.11 Slim
<i>Framework</i>	FastAPI + Uvicorn
<i>Container Port</i>	8080
<i>Region</i>	asia-southeast2 (Jakarta)
<i>Deployment Method</i>	GitHub Actions (<i>Continuous Deployment</i>)
<i>Container Registry</i>	Google Artifact Registry
<i>Min / Max Instances</i>	0 / 10 (<i>scale-to-zero</i>)

Tabel IV.10 menyajikan spesifikasi *hosting* untuk layanan *database*.

Tabel IV.10 Spesifikasi *Hosting Database*

Aspek	Spesifikasi
<i>Platform</i>	Google Cloud SQL
<i>Database Engine</i>	PostgreSQL
<i>Nama Instance</i>	asd-db
<i>Region</i>	asia-southeast2 (Jakarta)
<i>Koneksi</i>	Cloud SQL Connector (via --add-cloudsql-instances)

Deployment dilakukan secara otomatis melalui integrasi dengan repositori Git. Setiap *push* ke *branch* utama akan memicu *pipeline* GitHub Actions yang mendeteksi perubahan pada direktori *frontend/* atau *backend/* secara terpisah. *Image Docker* di-*build* dan dikirim ke Google Artifact Registry, kemudian di-*deploy* ke layanan Cloud Run yang sesuai. Autentikasi ke Google Cloud menggunakan Workload Identity Federation sehingga tidak memerlukan *service account key* yang disimpan secara statis.

BAB V

IMPLEMENTASI WEBSITE PEMBELAJARAN ALGORITMA DAN STRUKTUR DATA BERBASIS *GUIDED* *LEARNING*

Bab ini membahas proses implementasi sistem Teman Belajar ASD berdasarkan hasil perancangan yang telah diuraikan pada Bab IV. Implementasi dilakukan secara bertahap, dimulai dari penyiapan lingkungan pengembangan, pembangunan komponen *frontend* dengan Next.js, hingga pengembangan *backend* berbasis FastAPI beserta integrasi OpenAI API sebagai mesin generasi konten adaptif. Seluruh hasil implementasi mengacu pada kebutuhan fungsional dan nonfungsional yang telah ditetapkan.

V.1 Lingkungan Implementasi

Pada pengembangan sistem Teman Belajar ASD dibutuhkan perangkat baik sisi perangkat keras maupun perangkat lunak untuk mendukung proses implementasi. Penjelasan lingkungan implementasi pada sisi perangkat keras yang digunakan dapat dilihat pada Tabel V.1.

Tabel V.1 Lingkungan Implementasi Perangkat Keras

Spesifikasi	Detail
Perangkat	Laptop MacBook Pro
Prosesor	Apple Silicon M1
RAM	16 GB
Penyimpanan Internal	SSD 512 GB

Selain lingkungan perangkat keras, implementasi sistem ini membutuhkan dukungan perangkat lunak yang terpasang pada perangkat keras. Penjelasan lingkungan implementasi pada sisi perangkat lunak yang digunakan dapat dilihat pada Tabel V.2.

Tabel V.2 Lingkungan Implementasi Perangkat Lunak

Spesifikasi	Detail
Sistem Operasi	macOS Sequoia 15
<i>Text Editor</i>	Visual Studio Code
Bahasa Pemrograman	Python 3.11, JavaScript (Node.js 20)
<i>Frontend Framework</i>	Next.js 16, React 19, Tailwind CSS 4
<i>Backend Framework</i>	FastAPI, SQLAlchemy, Pydantic
<i>Containerization</i>	Docker
<i>Repository</i>	GitHub
<i>Cloud Platform</i>	Google Cloud Platform (Cloud Run, Artifact Registry)
CI/CD	GitHub Actions

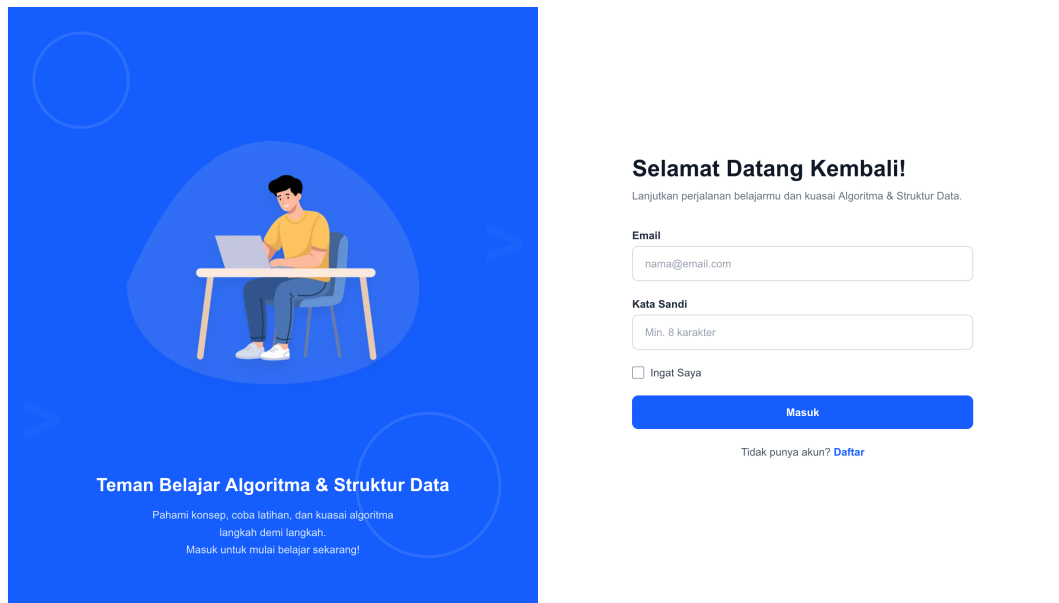
Dengan terpenuhinya kebutuhan lingkungan implementasi yang sesuai, implementasi sistem Teman Belajar ASD diharapkan dapat berjalan dengan lancar.

V.2 Implementasi *Frontend*

V.2.1 Implementasi Halaman *Login* dan Registrasi

Halaman *login* dan registrasi merupakan pintu masuk utama sistem yang menangani proses autentikasi pengguna. Kedua halaman menggunakan tata letak dua panel: panel kiri menampilkan ilustrasi dan deskripsi aplikasi, sedangkan panel kanan berisi formulir input. Implementasi autentikasi memanfaatkan fungsi-fungsi yang didefinisikan pada modul `app/lib/auth.js` sebagai lapisan abstraksi komunikasi dengan *backend*.

Gambar V.1 menunjukkan tampilan halaman *login* yang telah diimplementasikan.

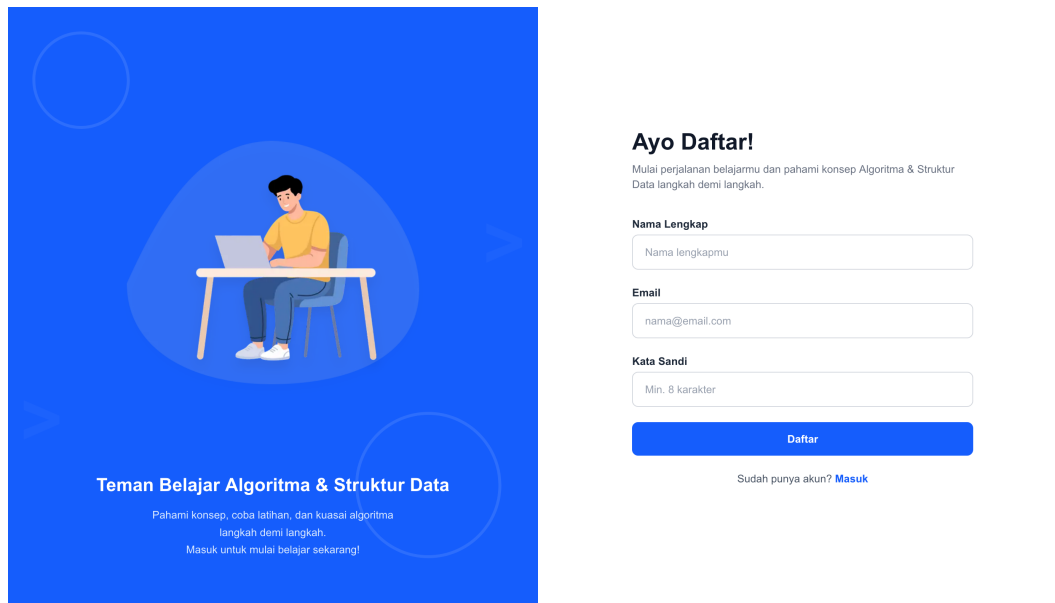


Gambar V.1 Tampilan Halaman *Login*

Formulir *login* memiliki dua *field* wajib, yaitu *email* dan kata sandi, keduanya divalidasi menggunakan atribut `required` dan `type="email"` bawaan HTML. Ketika pengguna mengirimkan formulir, fungsi `handleSubmit` mengirimkan permintaan POST `/api/auth/login` ke *backend*. Jika autentikasi berhasil, *backend* mengembalikan *JWT token* beserta data pengguna yang kemudian disimpan ke `localStorage` melalui fungsi `saveSession`. Sistem juga langsung memuat data progres belajar dari *database* ke `localStorage` (`loadProgressFromDb`) agar navigasi antartopik dapat berjalan tanpa latensi tambahan. Potongan kode implementasi alur *login* dapat dilihat pada Lampiran A, Kode A.1.

Sistem juga menerapkan proteksi halaman *login*: jika sesi aktif sudah tersimpan di `localStorage`, pengguna akan langsung dialihkan ke *dashboard* sehingga tidak perlu *login* ulang.

Gambar V.2 menunjukkan tampilan halaman registrasi yang diakses melalui tautan "Daftar" pada halaman *login*.



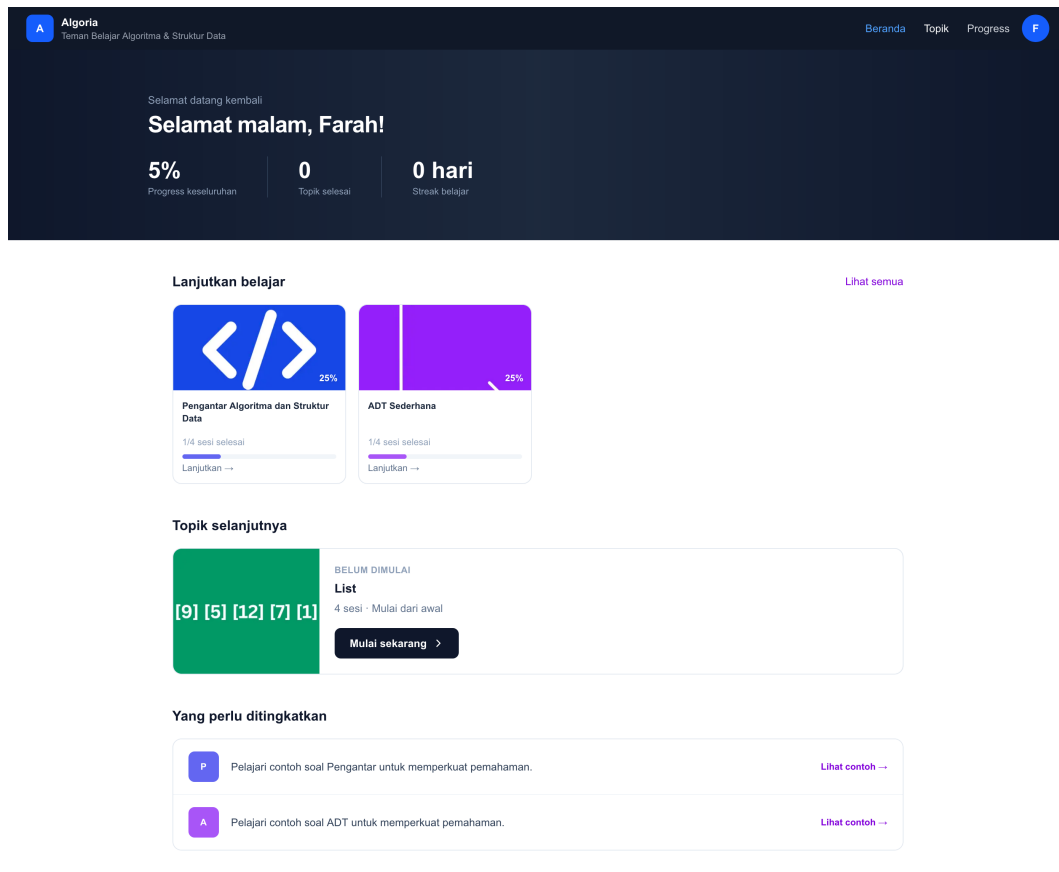
Gambar V.2 Tampilan Halaman Registrasi

Halaman registrasi memiliki formulir dengan tiga *field*: nama lengkap, *email*, dan kata sandi. Setelah registrasi berhasil, sistem secara otomatis menyimpan sesi dan mengalihkan pengguna ke *dashboard* tanpa perlu *login* kembali, menggunakan alur yang identik dengan *login* melalui fungsi `registerUser` pada modul `auth.js`.

V.2.2 Implementasi Halaman Beranda

Halaman beranda merupakan tampilan utama yang ditampilkan setelah pengguna berhasil masuk ke sistem. Halaman ini dirancang agar pengguna dapat langsung melihat ringkasan progres belajar, melanjutkan topik yang sedang dipelajari, mengetahui topik berikutnya yang perlu dimulai, serta mendapatkan rekomendasi topik yang perlu ditingkatkan.

Gambar V.3 menunjukkan tampilan halaman beranda yang telah diimplementasikan.



Gambar V.3 Tampilan Halaman Beranda

Bagian atas halaman menampilkan *hero banner* berlatar gelap yang memuat sapaan dinamis berdasarkan waktu akses (pagi, siang, atau malam) serta tiga statistik utama: persentase progres keseluruhan, jumlah topik yang telah diselesaikan, dan *streak* belajar harian. Nilai-nilai ini dihitung secara lokal dari data progres yang tersimpan di *localStorage*. Persentase keseluruhan diperoleh dari rata-rata progres seluruh 10 topik, di mana setiap topik memiliki empat sesi (materi, contoh, latihan, ringkasan) yang masing-masing bernilai 25%.

Di bawah *banner*, terdapat bagian "*Lanjutkan belajar*" yang menampilkan grid kartu topik yang sedang dalam proses (progres lebih dari 0% namun belum 100%). Setiap kartu memuat *thumbnail* topik, judul, jumlah sesi selesai, dan *progress bar* berwarna. Kartu-kartu ini diklik langsung menuju halaman topik yang bersangkutan. Bagian selanjutnya adalah "*Topik selanjutnya*", yang menampilkan satu topik pertama yang belum pernah dimulai dalam format kartu horizontal dengan *thumbnail* besar dan tombol "*Mulai sekarang*".

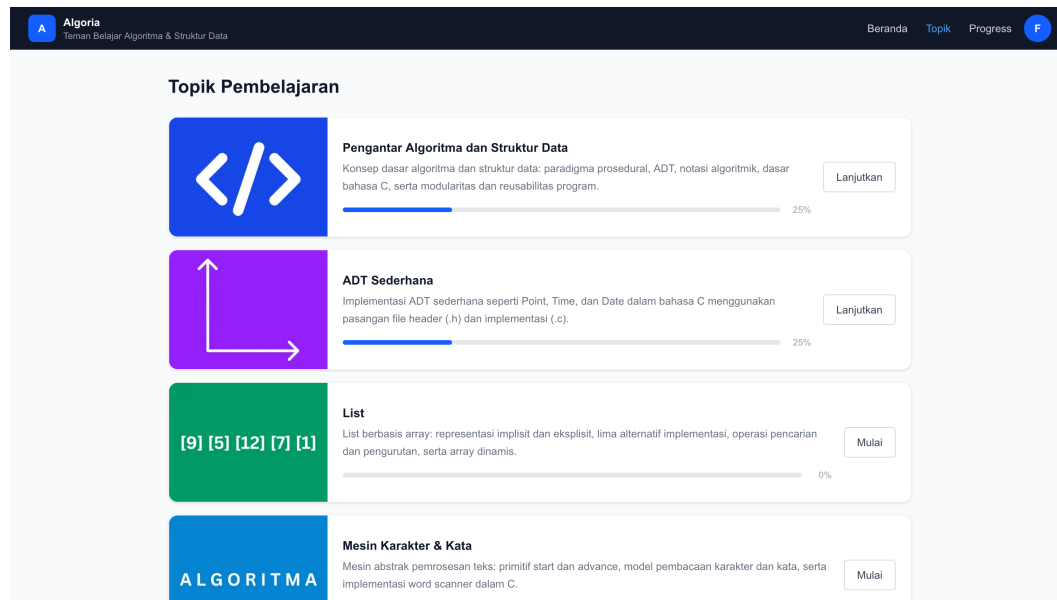
Bagian "*Yang perlu ditingkatkan*" menampilkan hingga tiga rekomendasi berdasarkan

analisis progres per sesi dari setiap topik. Rekomendasi dibangkitkan oleh fungsi `buildInsights` yang memprioritaskan topik dengan sesi hampir selesai, topik dengan materi terbaca tetapi latihan belum dikerjakan, serta topik yang baru dimulai. Setiap item rekomendasi memuat pesan kontekstual dan tautan aksi cepat menuju topik terkait. Potongan kode fungsi `buildInsights` dapat dilihat pada Lampiran A, Kode A.2.

V.2.3 Implementasi Halaman Topik

Halaman topik berfungsi sebagai katalog seluruh materi pembelajaran ASD yang tersedia. Halaman ini menampilkan 10 topik dalam format daftar *card* horizontal agar pengguna dapat melihat deskripsi singkat dan progres setiap topik secara sekilas.

Gambar V.4 menunjukkan tampilan halaman topik yang telah diimplementasikan.



Gambar V.4 Tampilan Halaman Topik

Setiap *card* terdiri dari tiga bagian utama. Pertama, *thumbnail* berwarna di sisi kiri yang memuat gambar ilustrasi khas topik dengan warna latar berbeda untuk masing-masing topik. Kedua, bagian informasi di tengah yang menampilkan judul topik, deskripsi singkat mencakup konsep-konsep kunci yang akan dipelajari, serta *progress bar* yang menunjukkan persentase penyelesaian topik tersebut. Ketiga, tombol aksi di sisi kanan yang secara dinamis bertuliskan "*Mulai*" jika topik belum pernah dibuka, atau "*Lanjutkan*" jika sudah ada progres.

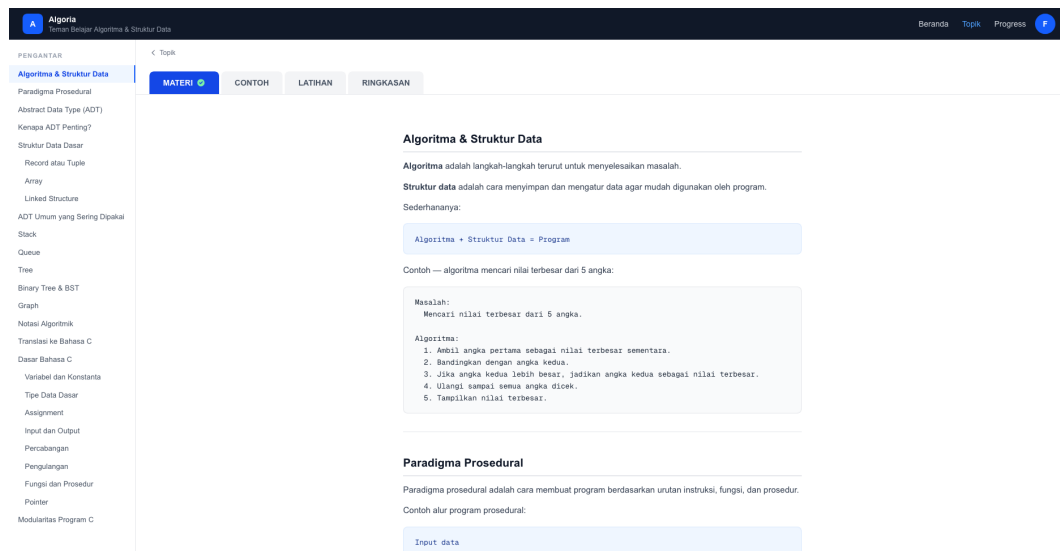
Nilai progres per topik dihitung dari `localStorage` dengan membaca status penye-

lesaian keempat sesi (materi, contoh, latihan, ringkasan). Setiap sesi yang bertanda selesai menambah 25% pada total progres topik tersebut. Pembacaan `localStorage` dilakukan pada saat komponen terpasang (*mount*) melalui `useEffect` sehingga nilai yang ditampilkan selalu mencerminkan kondisi terkini tanpa perlu permintaan ke *server*. Potongan kode perhitungan progres dan inisialisasi *state* dapat dilihat pada Lampiran A, Kode A.3.

V.2.4 Implementasi Halaman Materi

Halaman materi merupakan sesi pertama dalam alur pembelajaran terpandu setiap topik. Halaman ini menampilkan konten teori secara terstruktur dengan navigasi yang memudahkan pengguna menemukan subbagian tertentu.

Gambar V.5 menunjukkan tampilan halaman materi yang telah diimplementasikan.



Gambar V.5 Tampilan Halaman Materi

Tata letak halaman terdiri dari dua panel: panel kiri berupa *sidebar* navigasi berisi daftar isi (*table of contents*) yang hanya tampil pada layar desktop, dan panel kanan berupa area konten yang dapat digulir. Pada versi *mobile*, daftar isi dikemas dalam komponen *collapsible* yang dapat dibuka dengan menekan tombol di bagian atas konten. Saat pengguna menggulir konten, sorotan pada item daftar isi bergerak secara otomatis mengikuti bagian yang sedang terlihat menggunakan *event listener* pada elemen *scroll*.

Konten materi disusun menggunakan komponen-komponen presentasi yang telah didefinisikan, yaitu `CodeBlock` untuk kode program dengan sintaks berwarna, `Pseudocode`

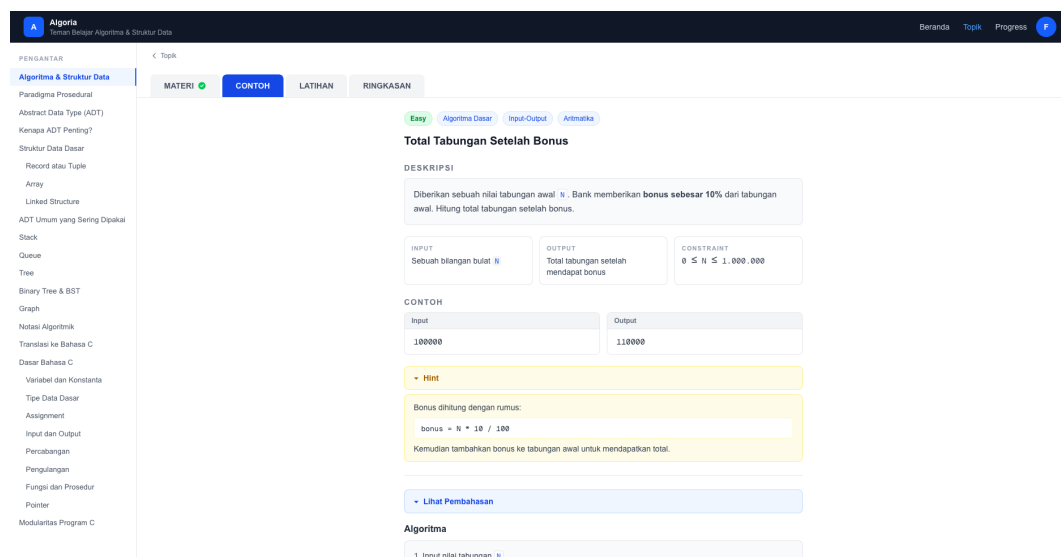
untuk notasi algoritmik, AsciiBox untuk diagram tekstual, W3Table untuk tabel perbandingan, dan NoteBox serta InfoBox untuk blok catatan penting.

Di bagian bawah setiap sesi terdapat tombol "*Tandai Selesai*" yang menyimpan status penyelesaian ke `localStorage` sekaligus menyinkronkan data ke *backend* melalui fungsi `syncTopicProgress`. Setelah sesi ditandai selesai, tombol digantikan dengan indikator centang hijau dan jumlah sesi yang telah diselesaikan diperbarui. Potongan kode fungsi penyimpanan progres dapat dilihat pada Lampiran A, Kode A.4.

V.2.5 Implementasi Halaman Contoh

Halaman contoh menyajikan satu soal latihan berbasis kasus dengan pembahasan lengkap. Tujuannya adalah memperlihatkan cara berpikir algoritmik secara bertahap sebelum pengguna mencoba mengerjakan soal sendiri di sesi latihan.

Gambar V.6 menunjukkan tampilan halaman contoh yang telah diimplementasikan.



Gambar V.6 Tampilan Halaman Contoh

Setiap soal contoh ditampilkan dalam format terstruktur: bagian atas menampilkan *badge* tingkat kesulitan dan kategori topik, diikuti judul soal, blok deskripsi masalah, serta tiga kartu ringkas yang menampilkan spesifikasi input, output, dan *constraint*. Di bawahnya terdapat contoh pasangan input-output dalam dua panel berdampingan.

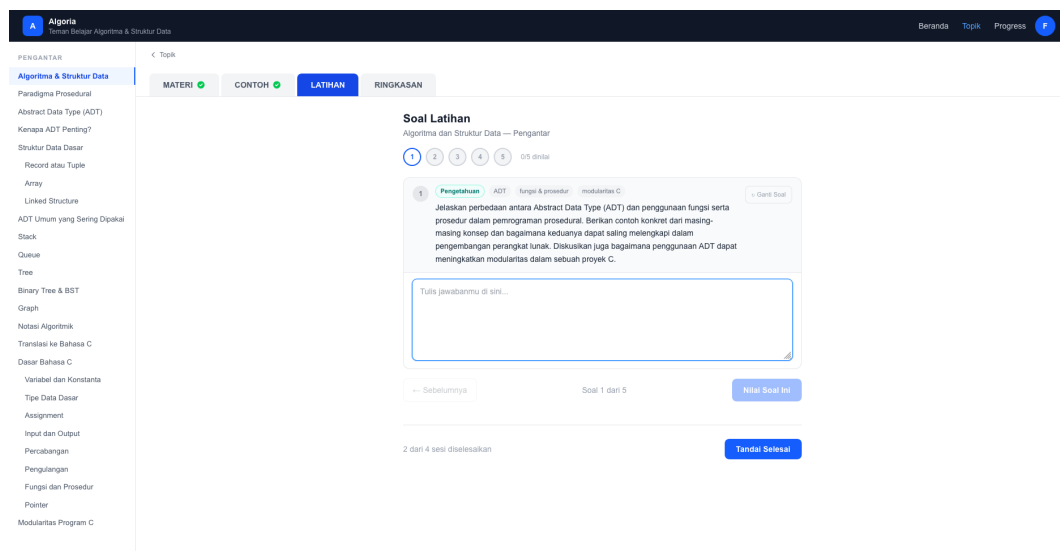
Halaman ini menerapkan dua komponen interaktif yang dapat diungkap secara terpisah: blok *Hint* yang memberikan petunjuk pendekatan solusi, dan blok *Lihat Pembahasan* yang menampilkan solusi lengkap mencakup algoritma langkah demi

langkah dalam bentuk daftar bernomor, notasi algoritmik dalam bentuk pseudocode, dan implementasi dalam bahasa C beserta penelusuran nilai (*trace*). Kedua blok ini tersembunyi secara *default* agar pengguna terdorong untuk memikirkan solusi terlebih dahulu sebelum melihat jawaban. Potongan kode implementasi mekanisme *toggle* tersebut dapat dilihat pada Lampiran A, Kode A.5.

V.2.6 Implementasi Halaman Latihan

Halaman latihan merupakan sesi inti sistem adaptif. Soal-soal latihan dibangkitkan secara dinamis oleh OpenAI GPT-4o-mini melalui *endpoint* POST `/api/latihan/generate` sehingga setiap sesi dapat menyajikan soal yang bervariasi dan disesuaikan dengan kelemahan pengguna.

Gambar V.7 menunjukkan tampilan halaman latihan yang telah diimplementasikan.



Gambar V.7 Tampilan Halaman Latihan

Setiap sesi latihan terdiri dari 5 soal dengan dua tipe: *pengetahuan* (jawaban esai singkat dengan textarea biasa) dan *implementasi* (jawaban kode C dengan textarea bertema gelap). Navigasi antara soal menggunakan *dot navigator* berwarna yang mencerminkan status setiap soal secara visual: abu-abu untuk belum dijawab, biru untuk sudah diisi, hijau untuk dinilai dengan skor ≥ 70 , dan merah untuk skor di bawah 70.

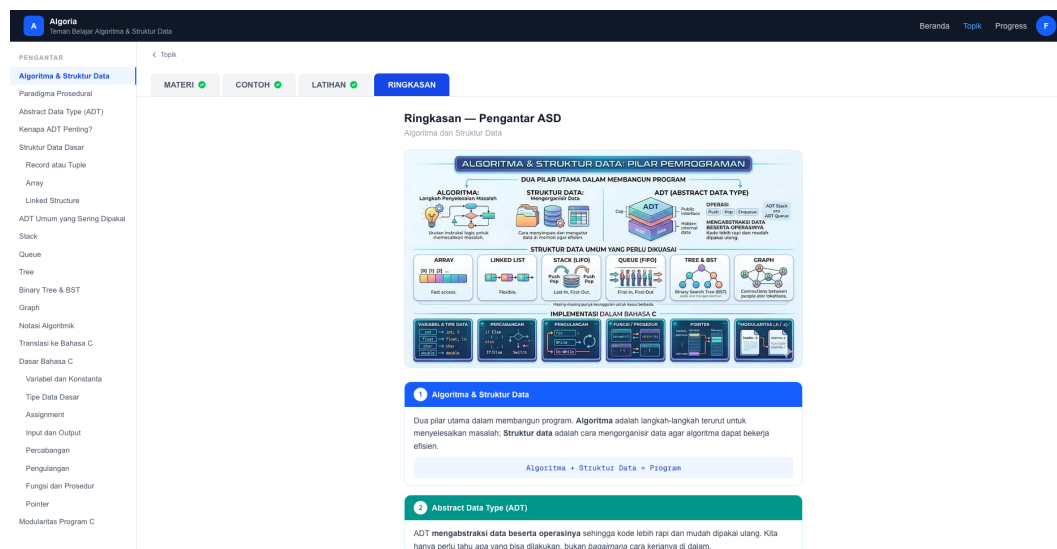
Setiap soal dievaluasi secara individual melalui *endpoint* POST `/api/latihan/evaluasi`. Hasil evaluasi meliputi skor 0–100, label nilai (Sangat Baik/Baik/Cukup/Perlu Perbaikan), *breakdown* metrik penilaian per aspek, komentar deskriptif, bagian yang sudah benar, bagian yang perlu diperbaiki, serta daftar konsep yang perlu dipelajari

ulang (konsep_lemah). Setelah semua soal selesai, halaman beralih ke tampilan ringkasan hasil yang merangkum rata-rata skor, analisis kelemahan, dan tombol untuk membangkitkan sesi soal baru yang difokuskan pada konsep-konsep lemah tersebut. Potongan kode alur *generate* dan evaluasi soal dapat dilihat pada Lampiran A, Kode A.6.

V.2.7 Implementasi Halaman Ringkasan

Halaman ringkasan merupakan sesi penutup dari setiap topik. Halaman ini menyajikan ulasan padat seluruh konsep yang telah dipelajari dalam format visual yang mudah diingat, sehingga berfungsi sebagai referensi cepat setelah pengguna menyelesaikan materi, contoh, dan latihan.

Gambar V.8 menunjukkan tampilan halaman ringkasan yang telah diimplementasikan.



Gambar V.8 Tampilan Halaman Ringkasan

Struktur konten ringkasan terdiri dari tiga bagian utama. Pertama, infografis topik yang merangkum seluruh konsep dalam satu gambar visual. Kedua, empat kartu *takeaway* kunci dengan warna berbeda yang masing-masing menjelaskan satu konsep utama dari topik tersebut secara ringkas, dilengkapi ilustrasi pendukung berupa diagram ASCII atau tabel mini di dalam kartu. Ketiga, *quick cheat sheet* yang menampilkan perbandingan konsep secara berdampingan untuk memperkuat pemahaman terhadap perbedaan konsep yang sering tertukar.

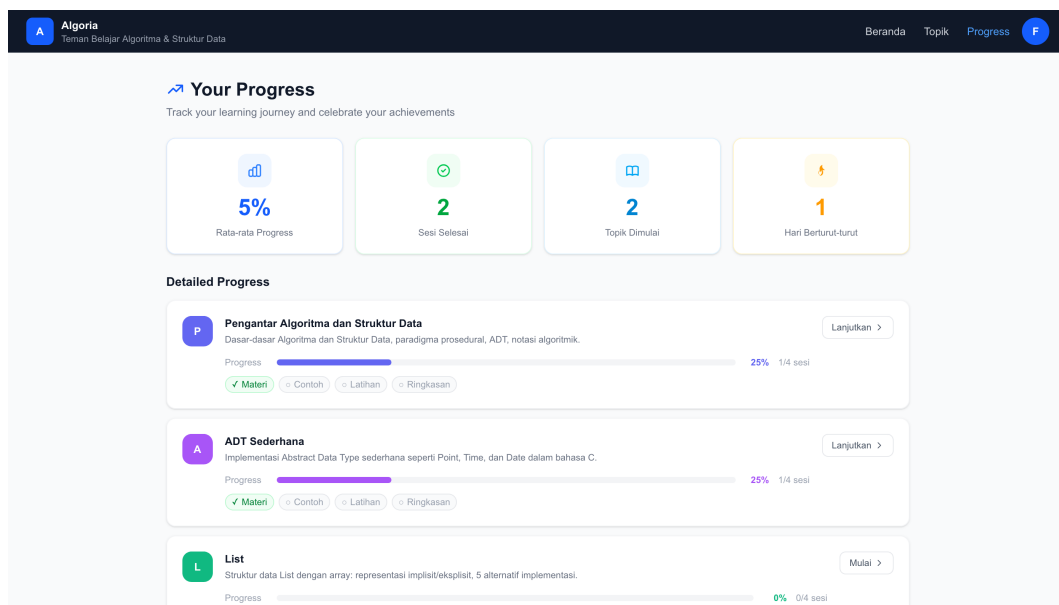
Halaman ringkasan menggunakan mekanisme penandaan sesi selesai yang identik dengan halaman materi dan contoh melalui fungsi `handleComplete` dan `saveProgress`.

Setelah keempat sesi (materi, contoh, latihan, dan ringkasan) ditandai selesai, progres topik mencapai 100% dan topik tersebut dianggap tuntas dalam sistem.

V.2.8 Implementasi Halaman *Progress*

Halaman *progress* menyediakan dasbor pemantauan kemajuan belajar secara menyeluruh. Pengguna dapat melihat ringkasan statistik keseluruhan sekaligus rincian status penyelesaian setiap topik dan sesinya dalam satu tampilan.

Gambar V.9 menunjukkan tampilan halaman *progress* yang telah diimplementasikan.



Gambar V.9 Tampilan Halaman *Progress*

Bagian atas halaman menampilkan empat kartu statistik ringkasan: rata-rata progres keseluruhan dalam persentase, total sesi yang telah diselesaikan dari seluruh topik, jumlah topik yang sudah pernah dimulai, dan jumlah hari belajar berturut-turut (*streak*). Keempat nilai ini dihitung secara lokal dari `localStorage` pada saat komponen di-*render* menggunakan `useSyncExternalStore`.

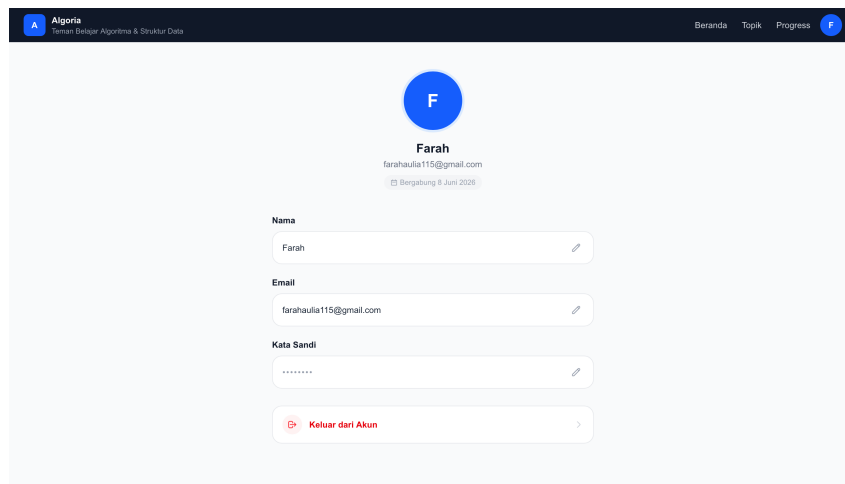
Di bawah kartu ringkasan terdapat daftar *Detailed Progress* yang menampilkan satu *TopicCard* per topik. Setiap kartu memuat ikon berwarna dengan inisial nama topik, judul, deskripsi singkat, *progress bar* dengan persentase dan jumlah sesi selesai, serta empat *badge* sesi (Materi, Contoh, Latihan, Ringkasan) yang berwarna hijau jika sesi tersebut telah diselesaikan dan abu-abu jika belum. Tombol "*Mulai*" atau "*Lanjutkan*" di sisi kanan mengarahkan pengguna langsung ke halaman topik yang bersangkutan.

Logika *streak* harian dihitung oleh fungsi `updateStreak` yang membandingkan tanggal akses hari ini dengan tanggal terakhir disimpan di `localStorage`. Jika selisihnya tepat satu hari, *streak* bertambah satu; jika lebih dari satu hari, *streak* direset ke 1. Nilai tanggal dan *streak* kemudian diperbarui di `localStorage`. Potongan kode fungsi `updateStreak` dapat dilihat pada Lampiran A, Kode A.7.

V.2.9 Implementasi Halaman Pengaturan

Halaman pengaturan memungkinkan pengguna mengelola data akun mereka, meliputi perubahan nama, *email*, dan kata sandi, serta melakukan *logout* dari sistem.

Gambar V.10 menunjukkan tampilan halaman pengaturan yang telah diimplementasikan.



Gambar V.10 Tampilan Halaman Pengaturan

Bagian atas halaman menampilkan avatar pengguna berupa lingkaran berwarna biru dengan inisial huruf pertama nama, disertai nama lengkap, *email*, dan tanggal bergabung yang diformat menggunakan `toLocaleDateString`.

Di bawah avatar terdapat tiga baris yang dapat diedit secara *inline*: nama, *email*, dan kata sandi. Setiap baris menampilkan nilai saat ini dalam mode hanya-baca (*read-only*) dengan ikon pensil di sisi kanan. Ketika ikon pensil ditekan, baris tersebut beralih ke mode edit yang menampilkan *input field* langsung di tempat tanpa berpindah halaman. Mode edit dikendalikan oleh satu *state* editing yang bernilai 'name', 'email', 'password', atau null. Perubahan dikirim ke *backend* melalui `PUT /api/auth/profile` menggunakan fungsi `updateProfile` dari modul `auth.js`.

Perubahan kata sandi divalidasi di sisi klien sebelum dikirim: kata sandi lama wajib diisi, kata sandi baru minimal 8 karakter, dan konfirmasi harus cocok. Sete-

lah penyimpanan berhasil, data sesi di `localStorage` diperbarui melalui fungsi `refreshSession` agar informasi yang tampil di seluruh halaman langsung mencerminkan perubahan tanpa perlu *login* ulang. Di bagian bawah terdapat tombol "*Keluar dari Akun*" yang memanggil `clearSession()` untuk menghapus sesi dari `localStorage` kemudian mengarahkan pengguna ke halaman *login*. Potongan kode fungsi `refreshSession` dan alur penyimpanan perubahan profil dapat dilihat pada Lampiran A, Kode A.8.

V.3 Implementasi *Backend*

Implementasi *backend* menggunakan FastAPI sebagai *framework* utama yang berjalan di atas Uvicorn. Seluruh *endpoint* dikelompokkan dalam satu `APIRouter` dan didaftarkan ke aplikasi utama dengan *prefix* `/api`. CORS dikonfigurasi agar hanya menerima permintaan dari *origin frontend* yang telah ditetapkan. Inisialisasi tabel *database* dilakukan secara otomatis saat aplikasi pertama kali dijalankan melalui *lifespan handler*.

V.3.1 Implementasi Autentikasi

Sistem autentikasi dibangun menggunakan dua mekanisme: PBKDF2-SHA256 untuk penyimpanan kata sandi dan JWT HS256 untuk sesi pengguna. Implementasi keduanya terpusat di modul `app/security.py`.

Kata sandi pengguna tidak pernah disimpan dalam bentuk teks biasa. Fungsi `hash_password` membangkitkan *salt* acak 16 byte menggunakan `secrets.token_hex`, kemudian menghasilkan *hash* PBKDF2-SHA256 dengan 100.000 iterasi. Hasil akhirnya disimpan dalam satu string berformat `pbkdf2_sha256$iterasi$salt$hash` sehingga semua parameter yang diperlukan untuk verifikasi tersedia tanpa kolom tambahan di *database*. Fungsi `verify_password` memisahkan kembali komponen tersebut, menghitung ulang *hash*, dan membandingkannya menggunakan `hmac.compare_digest` untuk mencegah *timing attack*.

JWT dibangkitkan oleh `create_access_token` dengan masa berlaku tujuh hari dan algoritma HS256. *Endpoint* yang memerlukan autentikasi menggunakan *dependency* `get_current_user` yang membaca *header* `Authorization: Bearer <token>`, memvalidasi dan mendekode JWT, lalu mengambil objek `User` dari *database*. Potongan kode implementasi *password hashing* dan `get_current_user` dapat dilihat pada Lampiran A, Kode A.9.

V.3.2 Implementasi API Progres

API progres menyediakan dua *endpoint*: `GET /api/progress` untuk mengambil seluruh data progres milik pengguna yang sedang *login*, dan `PUT /api/progress/{topic_slug}`

untuk menyimpan progres satu topik.

Endpoint PUT menerapkan pola *upsert*: sistem terlebih dahulu mencari baris *Progress* yang sesuai dengan *user_id* dan *topic_slug*. Jika belum ada, baris baru dibuat dan ditambahkan ke sesi *database*; jika sudah ada, nilainya diperbarui langsung. Kolom *last_accessed* selalu diperbarui ke waktu saat ini menggunakan `datetime.now(timezone.utc)`. Pola ini menghindari duplikasi baris sekaligus menghilangkan kebutuhan logika `INSERT OR UPDATE` yang berbeda-beda antar *database*. Potongan kode *endpoint* *upsert* progres dapat dilihat pada Lampiran A, Kode A.10.

V.3.3 Implementasi Mesin Soal Adaptif

Mesin soal adaptif diimplementasikan di modul `app/ai.py` dengan memanfaatkan OpenAI GPT-4o-mini melalui *endpoint* `/v1/chat/completions`. Modul ini mengekspos tiga fungsi utama: `generate_soal`, `evaluasi_soal`, dan `evaluasi_batch`, yang masing-masing dipanggil oleh *endpoint* `POST /api/latihan/{topic_slug}/generate`, `/evaluasi`, dan `/evaluasi-batch`.

Fungsi `generate_soal` menerima parameter kelemahan berisi daftar konsep yang perlu diperkuat dari sesi latihan sebelumnya. Jika kelemahan tidak kosong, instruksi sistem yang dikirim ke model menyertakan daftar konsep tersebut agar soal yang dibangkitkan berfokus pada area yang lemah. Setiap topik memiliki konfigurasi tersendiri di `TOPIC_CONFIGS` yang mendefinisikan nama, panduan kualitas soal, dan skema JSON yang harus dikembalikan oleh model. Respons model diurai dari JSON dan dikembalikan langsung sebagai *response* API.

Fungsi `evaluasi_soal` mengirimkan soal beserta jawaban pengguna ke model untuk dinilai. Model mengembalikan objek JSON yang memuat skor 0–100, label nilai, *breakdown* metrik penilaian per aspek, komentar deskriptif, bagian yang sudah benar, bagian yang perlu diperbaiki, dan daftar `konsep_lemah` yang digunakan sebagai masukan adaptif pada sesi latihan berikutnya. Potongan kode *endpoint* `generate` dan `evaluasi` dapat dilihat pada Lampiran A, Kode A.11.

BAB VI

EVALUASI

VI.1 Metode Evaluasi

Evaluasi Aplikasi Pembelajaran ASD dilakukan untuk memastikan bahwa sistem yang dikembangkan telah memenuhi seluruh kebutuhan fungsional yang ditetapkan, menghasilkan konten pembelajaran yang berkualitas, serta dapat digunakan dengan mudah oleh mahasiswa sebagai pengguna. Evaluasi terdiri atas dua bagian utama, yaitu pengujian *black box* untuk memverifikasi fungsionalitas sistem dan pengujian kegunaan (*usability testing*) untuk menilai pengalaman pengguna secara keseluruhan.

VI.1.1 Pengujian *Black Box*

Black Box Testing merupakan metode pengujian perangkat lunak yang mengevaluasi fungsi dan perilaku sistem dari sisi eksternal tanpa melihat struktur internal atau kode program, sehingga pengujian dilakukan dengan memberikan berbagai *input* dan memeriksa apakah *output* telah sesuai dengan spesifikasi yang ditentukan (Pressman dan Maxim 2015). Pada pengujian ini digunakan teknik *Equivalence Partitioning*, yaitu metode yang membagi domain *input* ke dalam beberapa kelas ekuivalen sehingga hanya satu nilai dari setiap kelas yang perlu diuji karena dianggap mewakili seluruh anggota kelas tersebut (Myers, Sandler, dan Badgett 2011).

Pengujian *black box* dilakukan terhadap seluruh fitur fungsional Aplikasi Pembelajaran ASD berdasarkan kebutuhan fungsional yang telah ditetapkan. Setiap fitur diuji dengan mendefinisikan skenario uji, data masukan, keluaran yang diharapkan, dan status kelulusan. Rangkuman cakupan pengujian *black box* dapat dilihat pada Tabel VI.1 berikut.

Tabel VI.1 Cakupan Pengujian *Black Box*

ID	Fitur	Skenario Uji	Indikator Keberhasilan
FT-01	Login	Pengguna memasukkan <i>email</i> dan <i>password</i> yang valid / tidak valid / kosong	Sistem memperbolehkan akses dengan kredensial valid; menampilkan pesan kesalahan untuk kredensial tidak valid atau kosong
FT-02	Logout	Pengguna menekan tombol <i>logout</i>	Sesi dihapus dan pengguna diarahkan ke halaman <i>login</i>
FT-03	Pilih Topik ASD	Pengguna memilih salah satu dari sepuluh topik yang tersedia	Sistem menampilkan halaman topik dengan tab sesuai status <i>guided flow</i> pengguna
FT-04	Tampilkan Materi	Pengguna membuka tab materi pada topik yang dipilih	Konten materi berhasil dihasilkan dan ditampilkan sesuai topik; progres diperbarui setelah selesai dibaca
FT-05	Tampilkan Contoh	Pengguna membuka tab contoh setelah menyelesaikan tab materi	Konten contoh berhasil dihasilkan dan ditampilkan; tab terkunci jika materi belum selesai
FT-06	Latihan Soal	Pengguna membuka tab latihan dan menerima soal yang di- <i>generate</i>	Soal latihan berhasil di- <i>generate</i> oleh LLM dan ditampilkan kepada pengguna
FT-07	Pembahasan Latihan	Pengguna mengumpulkan jawaban latihan	Sistem mengembalikan skor, <i>feedback</i> per soal, dan pembahasan yang relevan
FT-08	Ringkasan Materi	Pengguna membuka tab ringkasan setelah menyelesaikan latihan	Konten ringkasan berhasil di- <i>generate</i> dan ditampilkan; topik ditandai selesai

Bersambung ke halaman berikutnya

Tabel VI.1 Cakupan Pengujian *Black Box* (lanjutan)

ID	Fitur	Skenario Uji	Indikator Keberhasilan
FT-09	Progres Belajar	Pengguna membuka halaman progres belajar	Sistem menampilkan status penyelesaian seluruh topik beserta indikator kelemahan konsep
FT-10	<i>Generate Konten LLM</i>	Sistem mengirim <i>prompt</i> ke OpenAI API untuk menghasilkan materi, contoh, dan soal	OpenAI API mengembalikan konten dalam format JSON yang valid sesuai topik
FT-11	<i>Guided Learning Flow</i>	Pengguna mencoba mengakses tab yang belum terbuka	Sistem mengunci tab yang belum tersedia dan menampilkan notifikasi untuk menyelesaikan tahap sebelumnya
FT-12	<i>Feedback Jawaban</i>	Pengguna mengumpulkan jawaban latihan	Sistem menampilkan <i>feedback</i> spesifik per soal beserta skor yang akurat
FT-13	<i>Adaptive Learning Engine</i>	Pengguna menyelesaikan latihan dengan hasil tertentu	Sistem menyimpan <i>weak concepts</i> dan menggunakannya pada <i>generate</i> soal berikutnya
FT-14	Adaptasi Konten Pembelajaran	Pengguna dengan riwayat <i>weak concepts</i> membuka materi/contoh/latihan	Konten yang dihasilkan lebih menekankan konsep yang teridentifikasi sebagai kelemahan pengguna

VI.1.2 Pengujian Kegunaan (*Usability Testing*)

Selain pengujian fungsional, dilakukan pengujian kegunaan untuk menilai sejauh mana aplikasi dapat digunakan oleh mahasiswa dengan mudah, efisien, dan memuaskan. Pengujian kegunaan dilakukan melalui dua pendekatan, yaitu *task-based usability test* dan pengisian kuesioner *System Usability Scale* (SUS).

VI.1.2.1 *Task-Based Usability Test*

Task-based usability test dilakukan dengan meminta responden menyelesaikan serangkaian tugas yang mencerminkan alur penggunaan aplikasi secara nyata. Responden diminta untuk menyelesaikan tugas-tugas berikut secara berurutan tanpa bantuan

dari peneliti:

1. Melakukan *login* menggunakan akun yang telah disediakan.
2. Memilih salah satu topik ASD dari halaman *dashboard*.
3. Membaca konten materi dan menandai tab materi sebagai selesai.
4. Membuka tab contoh dan membaca contoh implementasi yang ditampilkan.
5. Mengerjakan latihan soal dan mengumpulkan seluruh jawaban.
6. Membaca pembahasan dan skor yang diberikan sistem.
7. Membuka tab ringkasan dan menyelesaikan alur belajar satu topik.
8. Mengakses halaman progres belajar dan melihat status penyelesaian topik.

Selama pengujian, peneliti mengamati dan mencatat waktu penyelesaian setiap tugas, kesalahan yang dilakukan, serta hambatan yang ditemui. Indikator keberhasilan *task-based usability test* adalah seluruh tugas dapat diselesaikan oleh responden dengan waktu yang wajar tanpa adanya kebingungan yang signifikan dalam mengikuti alur *guided learning*.

VI.1.2.2 *System Usability Scale (SUS)*

System Usability Scale (SUS) merupakan instrumen pengukuran kegunaan yang dikembangkan oleh John Brooke (1996) dan telah banyak digunakan dalam evaluasi sistem interaktif. SUS terdiri atas 10 pernyataan dengan skala Likert 1–5, di mana skor akhir dinormalisasi ke rentang 0–100. Interpretasi skor SUS mengacu pada ketentuan yang dapat dilihat pada Tabel VI.2.

Tabel VI.2 Interpretasi Skor SUS

Rentang Skor	Kategori	Keterangan
> 80,3	<i>Excellent</i>	Aplikasi sangat mudah digunakan
68 – 80,3	<i>Good</i>	Aplikasi mudah digunakan
51 – 68	<i>OK</i>	Aplikasi cukup dapat digunakan
< 51	<i>Poor</i>	Aplikasi sulit digunakan dan perlu perbaikan signifikan

Kuesioner SUS disebarikan kepada responden melalui Google Form setelah responden menyelesaikan *task-based usability test*. Target responden adalah mahasiswa yang sedang atau telah menempuh mata kuliah Algoritma dan Struktur Data. Indikator keberhasilan pengujian kegunaan adalah perolehan skor SUS minimal 68 yang menunjukkan bahwa aplikasi berada pada kategori *Good* dan layak digunakan sebagai media pembelajaran.

VI.1.2.3 Responden

Responden pengujian kegunaan merupakan mahasiswa yang sedang atau telah menempuh mata kuliah Algoritma dan Struktur Data. Kriteria responden ditetapkan sebagai berikut:

1. Mahasiswa aktif yang sedang atau telah menempuh mata kuliah Algoritma dan Struktur Data.
2. Memiliki pengalaman menggunakan perangkat komputer atau *smartphone* untuk keperluan akademik.
3. Bersedia mengikuti seluruh tahapan pengujian secara mandiri.

Pengujian dilakukan terhadap minimal 5 responden sesuai dengan panduan Nielsen (1993) yang menyatakan bahwa 5 pengguna sudah cukup untuk mengidentifikasi sebagian besar permasalahan kegunaan dalam sebuah antarmuka.

VI.2 Hasil Evaluasi

VI.2.1 Hasil Pengujian *Black Box*

VI.2.2 Hasil Pengujian Kegunaan

VI.3 Pembahasan Hasil Evaluasi

BAB VII

PENUTUP

VII.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan evaluasi yang telah dilakukan, dapat ditarik kesimpulan sebagai berikut.

TODO

TODO

VII.2 Saran

Terdapat beberapa saran untuk pengembangan sistem Teman Belajar ASD lebih lanjut.

TODO

TODO

TODO

DAFTAR PUSTAKA

- Cormen, Thomas H., Charles E. Leiserson, Ronald L. Rivest, dan Clifford Stein. 2009. *Introduction to Algorithms*. 3rd. Cambridge, Massachusetts: The MIT Press.
- Deng. 2025. *TODO: Judul lengkap artikel LLM untuk tugas pemrograman*. *TODO: journal/conference*, volume, halaman, DOI.
- et al., Jamie. 2024. *TODO: Judul lengkap artikel ChatGPT sebagai Teaching Assistant DSA*. *TODO: journal/conference*, volume, halaman, DOI.
- Goodrich, Michael T., dan Roberto Tamassia. 1998. *Data Structures and Algorithms in Java*. New York: John Wiley & Sons.
- Kasneci, Enkelejda, Kathrin Seßler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, dkk. 2023. “ChatGPT for good? On opportunities and challenges of large language models for education”. *Learning and Individual Differences* 103:102274. <https://doi.org/10.1016/j.lindif.2023.102274>.
- Leinonen, Juho, Arto Hellas, Sami Sarsa, Paul Denny, James Prather, dan Brett A. Becker. 2023. “Using Large Language Models to Enhance Programming Error Messages”. Dalam *Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE '23)*, vol. 1. ACM. <https://doi.org/10.1145/3545945.3569785>.
- Minaee, Shervin. 2025. *TODO: Judul lengkap artikel tentang Prompt Engineering*. *TODO: journal/arXiv/conference details*.
- Mtaho, Adam B., dan Leonard J. Mselle. 2024. “Difficulties in Learning the Data Structures Course: Literature Review”. *The Journal of Informatics* 4 (1). <https://doi.org/10.59645/tji.v4i1.136>.
- Myers, Glenford J., Corey Sandler, dan Tom Badgett. 2011. *The Art of Software Testing*. 3rd. Hoboken, NJ: John Wiley & Sons.

- Pressman, Roger S., dan Bruce R. Maxim. 2015. *Software Engineering: A Practitioner's Approach*. 8th. New York: McGraw-Hill Education.
- Raihan, Nishat, Mohammed Latif Siddiq, Joanna C.S. Santos, dan Marcos Zampieri. 2024. *Large Language Models in Computer Science Education: A Systematic Literature Review*. Accepted at SIGCSE TS 2025. <https://doi.org/10.48550/arXiv.2410.16349>. arXiv: 2410.16349 [cs.LG].
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, dan Illia Polosukhin. 2017. "Attention Is All You Need". Dalam *Advances in Neural Information Processing Systems*, 30:5998–6008.
- White, Jules, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, dan Douglas C. Schmidt. 2023. *A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT*. arXiv: 2302.11382 [cs.SE].
- Zaus. 2025. *TODO: Judul lengkap artikel chatbot untuk instruksi algoritma*. TODO: journal/conference, volume, halaman, DOI — periksa juga nama penulis (teks menyebut Mahesi et al.)

LAMPIRAN A

KODE PROGRAM

Pada umumnya, kode program tidak perlu disertakan di dalam laporan tugas akhir karena kode program biasanya sangat panjang dan sudah disimpan dalam bentuk berkas-berkas elektronik terpisah di repositori daring. Namun, bagian kode program singkat yang penting untuk memahami implementasi sistem disertakan berikut ini.

A.1 Kode Program *Frontend*

```
1  const handleSubmit = async (e) => {
2    e.preventDefault();
3    setLoading(true);
4    try {
5      const data = await loginUser(form.email, form.password);
6      saveSession(data);
7      await loadProgressFromDb(data.token);
8      router.push('/dashboard');
9    } catch (err) {
10     setError(err.message);
11   } finally {
12     setLoading(false);
13   }
14 };
15
16 // auth.js
17 export function saveSession(data) {
18   localStorage.setItem('token', data.token);
19   localStorage.setItem('user', JSON.stringify(data.user));
20 }
```

Listing A.1 Implementasi Alur *Login* dan Penyimpanan Sesi

```
1  function buildInsights(rows) {
2    const list = [];
3    rows.forEach(({ topic, d, secs }) => {
4      if (secs === 0) return;
5      if (secs === 3) {
6        const missing = SECTIONS.find((s) => !d[s]);
```

```

7     list.push({ priority: 1, topic,
8       message: `Hampir selesai di ${topic.shortTitle} - tinggal
9       ${SECTION_LABELS[missing]}.`,
10      cta: 'Selesaikan' });
11   } else if (d.materi && d.contoh && !d.latihan) {
12     list.push({ priority: 2, topic,
13       message: `Sudah baca materi ${topic.shortTitle}, tapi
14       belum mencoba latihan.`,
15      cta: 'Coba latihan' });
16   } else if (d.materi && !d.contoh) {
17     list.push({ priority: 3, topic,
18       message: `Pelajari contoh soal ${topic.shortTitle} untuk
19       memperkuat pemahaman.`,
20      cta: 'Lihat contoh' });
21   }
22 }
23 return list.sort((a, b) => a.priority - b.priority).slice(0, 3);
24 }

```

Listing A.2 Fungsi buildInsights untuk Rekomendasi Topik

```

1 function calcProgress(key) {
2   try {
3     const d = JSON.parse(localStorage.getItem(key)) ?? {};
4     return ['materi', 'contoh', 'latihan', 'ringkasan']
5       .filter((s) => d[s]).length * 25;
6   } catch { return 0; }
7 }
8
9 // Inisialisasi saat komponen mount
10 useEffect(() => {
11   setPengantarProgress(calcProgress('asd_progress_pengantar'));
12   setAdtSederhanaProgress(calcProgress('asd_progress_adt_sederhana
13     '));
14   // ... topik lainnya
15 }, []);

```

Listing A.3 Perhitungan Progres Topik dari localStorage

```

1 function saveProgress(updates) {
2   const data = { ...readProgress(), ...updates };
3   localStorage.setItem(PROGRESS_KEY, JSON.stringify(data));
4   syncTopicProgress(TOPIC_SLUG, data);
5 }
6
7 const handleComplete = (tab) => {

```

```

8  const key = TAB_KEYS[tab];           // 'materi' | 'contoh' | '
    latihan' | 'ringkasan'
9  if (!key || completed[key]) return;
10 saveProgress({ [key]: true });
11 completedRef.current = { ...completedRef.current, [key]: true };
12 forceUpdate((n) => n + 1);
13 };

```

Listing A.4 Fungsi Penyimpanan Progres Sesi ke localStorage dan Backend

```

1  const [showHint, setShowHint]      = useState(false);
2  const [showJawaban, setShowJawaban] = useState(false);
3
4  // Tombol Hint
5  <button onClick={() => setShowHint(!showHint)}>
6    {showHint ? " " : " "} Hint
7  </button>
8  {showHint && <div>...petunjuk pendekatan...</div>}
9
10 // Tombol Pembahasan
11 <button onClick={() => setShowJawaban(!showJawaban)}>
12   {showJawaban ? " " : " "} Lihat Pembahasan
13 </button>
14 {showJawaban && <div>...algoritma, notasi, kode C, trace...</div>}

```

Listing A.5 Mekanisme Toggle Hint dan Pembahasan pada Halaman Contoh

```

1  const generateSoal = useCallback(async (kelemahan = []) => {
2    setFase("loading");
3    const res = await fetch("/api/latihan/generate", {
4      method: "POST",
5      body: JSON.stringify({ jumlah: 5, kelemahan }),
6    });
7    const data = await res.json();
8    setSoalList(data.soal);
9    setFase("latihan");
10 }, []);
11
12 const evaluasiSoal = async () => {
13   const res = await fetch("/api/latihan/evaluasi", {
14     method: "POST",
15     body: JSON.stringify({ soal, jawaban: jawaban[soal.id] }),
16   });
17   const data = await res.json();
18   setFeedbackMap((prev) => ({ ...prev, [soal.id]: data }));
19 };

```

```

20
21 // Saat generate baru, kirim konsep lemah dari sesi sebelumnya
22 const handleGenerateBaru = () => {
23   const kelemahan = [
24     ...new Set(Object.values(feedbackMap).flatMap((f) => f.
25       konsep_lemah ?? [])),
26   ];
27   generateSoal(kelemahan);
28 };

```

Listing A.6 Alur Generate Soal Adaptif dan Evaluasi per Soal

```

1 function updateStreak() {
2   const today = new Date().toISOString().split('T')[0];
3   const lastVisit = localStorage.getItem('asd_last_visit');
4   let streak = parseInt(localStorage.getItem('asd_streak') ?? '0',
5     10);
6   if (!lastVisit) {
7     localStorage.setItem('asd_last_visit', today);
8     localStorage.setItem('asd_streak', '1');
9     return 1;
10  }
11  if (lastVisit === today) return streak || 1;
12  const diff = Math.round(
13    (new Date(today) - new Date(lastVisit)) / 86400000
14  );
15  streak = diff === 1 ? streak + 1 : 1;
16  localStorage.setItem('asd_last_visit', today);
17  localStorage.setItem('asd_streak', String(streak));
18  return streak;
19 }

```

Listing A.7 Fungsi Perhitungan *Streak* Belajar Harian

```

1 const refreshSession = (updatedUser) => {
2   const s = getSession();
3   const newUser = { ...s.user, ...updatedUser };
4   saveSession({ token: s.token, user: newUser }); // perbarui
5   localStorage
6   setSession((prev) => ({ ...prev, user: newUser }));
7 };
8
9 const saveName = async () => {
10   if (!nameDraft.trim())
11     return setStatus({ field: 'name', error: 'Nama tidak boleh
12       kosong.' });
13 }

```



```

11  setLoading(true);
12  try {
13      const updated = await updateProfile(session.token, { name:
        nameDraft.trim() });
14      refreshSession(updated);
15      setEditing(null);
16  } catch (err) {
17      setStatus({ field: 'name', error: err.message });
18  } finally { setLoading(false); }
19  };

```

Listing A.8 Fungsi refreshSession dan Alur Penyimpanan Perubahan Profil

A.2 Kode Program Backend

```

1  # security.py
2  def hash_password(password: str) -> str:
3      salt = secrets.token_hex(PBKDF2_SALT_BYTES) # 16 byte acak
4      h = hashlib.pbkdf2_hmac("sha256", password.encode(),
5                               salt.encode(), 100_000).hex()
6      return f"pbkdf2_sha256$100000${salt}${h}"
7
8  # routes.py
9  def get_current_user(
10     authorization: str = Header(...),
11     db: Session = Depends(get_db),
12 ) -> User:
13     token = authorization.removeprefix("Bearer ")
14     payload = decode_access_token(token, settings.secret_key)
15     user_id = int(payload["sub"])
16     user = db.execute(
17         select(User).where(User.user_id == user_id)
18     ).scalar_one_or_none()
19     if user is None:
20         raise HTTPException(status_code=404, detail="Pengguna
        tidak ditemukan.")
21     return user

```

Listing A.9 Implementasi Password Hashing dan Dependency Autentikasi

```

1  @router.put("/progress/{topic_slug}", response_model=
    ProgressResponse)
2  async def upsert_progress(
3      topic_slug: str,
4      payload: ProgressUpdateRequest,
5      current_user: User = Depends(get_current_user),
6      db: Session = Depends(get_db),

```

```

7 ) -> ProgressResponse:
8     row = db.execute(
9         select(Progress).where(
10             Progress.user_id == current_user.user_id,
11             Progress.topic_slug == topic_slug,
12         )
13     ).scalar_one_or_none()
14
15     if row is None:                                # belum ada - buat
16         row = Progress(user_id=current_user.user_id, topic_slug=
17             topic_slug)
18         db.add(row)
19
20     row.materi      = payload.materi                # perbarui status 4
21     row.contoh      = payload.contoh
22     row.latihan     = payload.latihan
23     row.ringkasan   = payload.ringkasan
24     row.last_accessed = datetime.now(timezone.utc)
25     db.commit()
26     db.refresh(row)
27     return ProgressResponse.model_validate(row)

```

Listing A.10 Implementasi *Upsert* Progres Belajar per Topik

```

1 @router.post("/latihan/{topic_slug}/generate")
2 async def generate_latihan(
3     topic_slug: str, payload: GenerateSoalRequest,
4 ) -> dict:
5     result = await generate_soal(
6         topic_slug=topic_slug,
7         jumlah=payload.jumlah,
8         kelemahan=payload.kelemahan,                # konsep lemah dari sesi
9         sebelumnya
10         tipe_paksa=payload.tipe_paksa,
11         topik_referensi=payload.topik_referensi,
12     )
13     return result
14
15 @router.post("/latihan/{topic_slug}/evaluasi")
16 async def evaluasi_latihan(
17     topic_slug: str, payload: EvaluasiSoalRequest,
18 ) -> dict:
19     # model mengembalikan: skor, nilai, metrik, komentar,
20     konsep_lemah

```

```
19     return await evaluasi_soal(  
20         topic_slug=topic_slug,  
21         soal=payload.soal,  
22         jawaban=payload.jawaban,  
23     )
```

Listing A.11 Implementasi Endpoint Generate dan Evaluasi Soal Adaptif

LAMPIRAN B

HASIL SURVEI LAPANGAN

B.1 Foto Survei Lokasi 1

Teks survei lokasi 1.

B.2 Foto Survei Lokasi 2

Teks survei lokasi 2.

B.3 Transkrip Wawancara dengan Narasumber

Teks transkrip wawancara.

